Robust Model Predictive Control for Modular Multilevel Converter Current Control Toolbox

User Guide and Reference Manual

Victor Daniel Reyes Dreke, Rahul Rane, and Aleksandra Lekić
IEPG / TU Delft

Version 1.0
February 17, 2026

Contents

1 Introduction **3**
1.1 Purpose of the Toolbox . . . 3
1.2 Scope and Audience . . . 3

2 Installation and Requirements **3**
2.1 System Requirements . . . 3
2.2 Installation . . . 3

3 Mathematical Background **4**
3.1 MMC Model . . . 4
3.1.1 Output Current Model . . . 4
3.1.2 Circulating Current Model . . . 5
3.1.3 Zero-sequence Current Model . . . 5
3.2 Steady-state realizations . . . 5
3.2.1 Continuous time LTI and LPV model . . . 6
3.2.2 Discrete time LTI and LPV model . . . 7
3.2.3 Augmented model with integral action . . . 7
3.3 MMC Control Problem . . . 7
3.4 MPC Problem Formulation . . . 8
3.4.1 QP Programming of MPC controller . . . 9
3.5 State feedback controller synthesis . . . 10
3.5.1 Linear Robust/Quadratic Design . . . 10
3.5.2 Linear Constrained Robust Design . . . 11
3.5.3 Linear Optimal Robust Design [1, 2] . . . 11
3.6 MPC Tuning via Controller Matching . . . 11

4 Controller Architecture **11**

5 Getting Started **11**
5.1 Basic Workflow . . . 11
5.1.1 Basic workflow for defining system dynamics . . . 12
5.1.2 Basic workflow for designing matching controller and MPC tuning . . . 12
5.1.3 Basic workflow for creating MPC structure . . . 14
5.1.4 MPCClass Class . . . 17

6 Toolbox Architecture **17**
6.1 Folder Structure . . . 17

7 MATLAB Function Reference **17**
7.1 System dynamics definition . . . 17
7.1.1 `getPlantModel` . . . 17
7.1.2 `getPlantModelPU` . . . 18
7.1.3 `getAugmentedSystem` . . . 18
7.2 Synthesis state feedback controller . . . 19
7.2.1 `getSteadyState` . . . 19
7.2.2 `getRobustK` . . . 19
7.3 MPC tuning via controller matching . . . 20
7.3.1 `matchingControllerDesignRelax` . . . 20
7.4 MPC Formulation . . . 20
7.4.1 `ABN2PhiGamma` . . . 20
7.4.2 `QRPN2PsiOmega` . . . 21
7.4.3 `getWLc` . . . 21
7.4.4 `getMb` . . . 22
7.4.5 `getAXbx` . . . 22
7.4.6 `getAyby` . . . 22
7.4.7 `getAUbu` . . . 23
7.4.8 `getAdUbdu` . . . 23
7.4.9 `getAiUbiu` . . . 23
7.4.10 `QPHildClassic` . . . 24

8 Examples **25**
8.1 Example: Matching controller computation . . . 25
8.2 Example: Robust MPC vs Classic MPC . . . 28
8.3 Example: Modeling 2GW HVDC station from Tennet . . . 32
8.3.1 Uncertain output current model . . . 33
8.3.2 Output current controller design . . . 36
8.4 Example: Unconstrained control of 2GW HVDC station from Tennet . . . 40

9 Appendix A: Basic flowchart of the workflow of the toolbox **44**

10 References **45**

1 Introduction

This document describes a **toolbox to design robust model predictive control (rMPC) for modular multilevel converters (MMC)** that can be deployed using **MATLAB**©, **Julia**© and **RSCAD**©. The focus of the **rMPC** toolbox is to provide the user with a set of tools to tune the MPC such that it is **guaranteed robust performance against faults and modeling mismatches**. **The results exhibited here add to existing theory of MPC controllers design for linear time invariant (LTI) and linear parameter varying (LPV) systems.** This manual describes a collection of functions and scripts designed to formulate and solve finite optimal control problems (OCP) for the above-mentioned control problem, including:

- Synthesis of **robust-optimal** state feedback gain controller (oSFC)
- Synthesis of **constrained** state feedback gain controller (cSFC)
- Model predictive control (MPC)-type formulations
- MPC tuning via matching techniques: **robust** and **quadratic** stability guarantees.

1.1 Purpose of the Toolbox

The toolbox provides a high-level interface to design a rMPC solution for any system with by LTI or LPV dynamics. More specifically, it is tailored to control output and circulating currents of MMC, such that:

- Develop state-space realization of the MMC currents' dynamics using LTI and LPV models
- Generate of MPC ingredients: cost function tuning matrices (Q,R,P) and terminal constraints
- Easily integrate with power hardware in the loop (PHIL), real-time simulation (RSCAD), and visualization tools (MATLAB, and Julia)

1.2 Scope and Audience

This manual is intended for:

- Engineers and researchers in control and power system fields
- Students in optimal control, MPC, and power system courses
- Developers creating custom controllers for dynamic systems

The reader is assumed to be familiar with basic optimal control theory, power systems, MMC modeling and MATLAB programming.

2 Installation and Requirements

2.1 System Requirements

- MATLAB R2011a or later
- Julia v1.12.1 or later
- YALMIP: A Toolbox for Modeling and Optimization in `MATLAB` [3]
- MOSEK Solver [3]

Optional dependencies may include:

- The Multi-Parametric Toolbox (`MPT3`) [4]

2.2 Installation

1. Download or clone the toolbox repository into a folder: `rmpc_mmc_toolbox lib`
2. Add the folder to the MATLAB path (e.g., via `Set Path` or programmatically)

```
addpath('\path\to\rmpc_mmc_toolbox\lib');
savepath;
```

3 Mathematical Background

3.1 MMC Model

We analyze the MMC current dynamics divided into three groups: (i) output currents, (ii) circulating currents, and (iii) zero-sequence current. **All the dynamics are studied in the dqz-framework and consider possible modeling mismatches**.

The following signals, parameters and passive components are defined the same across all the MMC current dynamics groups:

- The amplitude of AC-link voltages are V_d^g and V_q^g.
- The amplitude of DC-link voltage is V_{dc}
- The angular velocity is $\omega = 2\pi f$ with f being the fundamental frequency of the grid.
- The arm inductance is $L_m = L_{m0} + \Delta L_m$, where $(L_{m0}, \Delta L_m)$ represents the **known nominal value** and **unknown uncertain value**, respectively.
- The arm resistance is $R_m = R_{m0} + \Delta R_m$, where $(R_{m0}, \Delta R_m)$ represents the known nominal value and unknown uncertain value, respectively.
- The AC-side inductance is $L_g = L_{g0} + \Delta L_g$, $(L_{g0}, \Delta L_g)$ represents the known nominal value and unknown uncertain value, respectively.
- The AC-side resistance is $R_g = R_{g0} + \Delta R_g$, $(R_{g0}, \Delta R_g)$ represents the known nominal value and unknown uncertain value, respectively.

Remark 3.1. *The **unknown uncertain values** of passive components refers primarily to their tolerances, which define the range within which the actual value of a component may deviate from its nominal (rated) value. These deviations arise due to factors such as temperature variations, aging effects, core material properties, and manufacturing precision. Note that other elements can be considered in the unknown uncertain values, e.g., discretization errors.*

Remark 3.2. *The set of differential equations here presented needs to be updated to consider the direction of the currents in each application. Here, we consider the currents circulating from the DC-link to the onshore AC-link (Grid).*

Readers are referred to [5] for an in-depth explanation of the MMC model. The explanation in [5] can be summarized as follows.

3.1.1 Output Current Model

The output currents of the MMC, i_d^Δ and i_q^Δ, have the following dynamics:

$$\frac{d}{dt} i_d^\Delta = -\frac{R_g^{eq}}{L_g^{eq}} i_d^\Delta - \omega i_q^\Delta + \frac{1}{L_g^{eq}} u_d^\Delta - \frac{1}{L_g^{eq}} V_d^g, \tag{1a}$$

$$\frac{d}{dt} i_q^\Delta = -\frac{R_g^{eq}}{L_g^{eq}} i_q^\Delta + \omega i_d^\Delta + \frac{1}{L_g^{eq}} u_q^\Delta - \frac{1}{L_g^{eq}} V_q^g, \tag{1b}$$

with

$$L_g^{eq} = L_g + \frac{L_m}{2}, \quad \text{and} \quad R_g^{eq} = R_g + \frac{R_m}{2}, \tag{2}$$

where the **modeling mismatches are considered by** defining the passive components values, such that

$$\begin{aligned} L_m &= L_{m0} + \Delta L_m, \quad & R_m &= R_{m0} + \Delta R_m, \\ L_g &= L_{g0} + \Delta L_g, & R_g &= R_{g0} + \Delta R_g. \end{aligned} \tag{3}$$

In this case, we define the **state vector** (x) comprised by the output currents (i_d^Δ, i_q^Δ) as $x = \begin{bmatrix} i_d^\Delta & i_q^\Delta \end{bmatrix}^\top$; the **control input vector** (u) comprised by the output voltages (u_d^Δ, u_q^Δ) as $u = \begin{bmatrix} u_d^\Delta & u_q^\Delta \end{bmatrix}^\top$; and the **exogenous input vector** (w) comprised by the output voltages (V_d^g, V_q^g) as $w = \begin{bmatrix} V_d^g & V_q^g \end{bmatrix}^\top$.

3.1.2 Circulating Current Model

The circulating currents of the MMC, i_d^Σ and i_q^Σ, have the following dynamics:

$$\frac{d}{dt} i_d^\Sigma = -\frac{R_m}{L_m} i_d^\Sigma + 2\omega i_q^\Sigma - \frac{1}{L_m} u_d^\Sigma, \tag{4a}$$

$$\frac{d}{dt} i_q^\Sigma = -\frac{R_m}{L_m} i_q^\Sigma - 2\omega i_d^\Sigma - \frac{1}{L_m} u_q^\Sigma. \tag{4b}$$

In this case, we define the **state vector** (x) comprised by the circulating currents (i_d^Σ, i_q^Σ) as $x = \begin{bmatrix} i_d^\Sigma & i_q^\Sigma \end{bmatrix}^\top$; and the **control input vector** (u) comprised by the circulating voltages (u_d^Σ, u_q^Σ) as $u = \begin{bmatrix} u_d^\Sigma & u_q^\Sigma \end{bmatrix}^\top$.

3.1.3 Zero-sequence Current Model

The zero-sequence current of the MMC, i_z^Σ, has the following dynamics:

$$\frac{d}{dt} i_z^\Sigma = -\frac{R_m}{L_m} i_z^\Sigma - \frac{1}{L_m} u_z^\Sigma + \frac{1}{2L_m} V_{dc}. \tag{5}$$

In this case, we define the **state** (x) comprised by the zero-sequence current (i_z^Σ); the **control input** (u) comprised by the zero-sequence voltage (u_z^Σ); and the **exogenous input** (w) comprised by the DC-link voltage (V_{dc}).

3.2 Steady-state realizations

In the following sections, the steady-state realizations of the MMC model are derived using the control inputs, states, and outputs defined in the previous section. Note that the MMC current groups exhibit decoupled dynamics yet it can be modeled as composite systems in which all currents are considered simultaneously. **In this toolbox, we analyze them separately.**
Regarding the modeling of uncertainties, the following distinctions are made:

- **LPV realizations are used to describe the system dynamics with uncertainties**. In this manual and toolbox, **all LPV realizations are affine representations**, where the nominal dynamics are denoted by the subscript 0, and the uncertain dynamics are denoted by the prefix Δ, such that

  $$\dot{x} = A(\rho)x \quad \text{with} \quad A(\rho) = A_0 + \Delta A(\rho)$$

  where $\rho(t) = \rho$, for all $t \geq 0$.

- **LPV realizations can describe applications where the nominal values changes with time** (in a piece-wise fashion), such that

  $$\dot{x} = A(\rho)x \quad \text{with} \quad A(\rho) = A_0(\rho) + \Delta A(\rho)$$

  where $\rho(t+1) = \rho(t)$, for all $t \geq 0$.

- **LTI realizations are used to describe the system when all model parameters are known** including when only the nominal dynamics ($\Delta A = 0$) are considered,

$$\dot{x} = Ax \quad \text{with} \quad A = A_0 + \Delta A$$

Remark 3.3. *For simplicity, any LPV realization presented has the following structure:*

$$\Xi(\rho) = \Xi_0 + \begin{bmatrix} \rho_{1,1} & \cdots & \rho_{1,b} \\ & & \\ \rho_{a,1} & \cdots & \rho_{a,b} \end{bmatrix} \tag{6}$$

where $\Xi_0 \in \mathbb{R}^{a\times b}$ is known matrix, $\rho = [\rho_{1,1}, ...\rho_{a,b}] \in \mathbb{P} \subset \mathbb{R}^{ab\times 1}$ is vector of scheduling variables with the origin in its interior, i.e., $0_{ab\times 1} \in \mathbb{P}$ and $(a, b) \in \mathbb{N}$ are the dimensions of the matrices.

3.2.1 Continuous time LTI and LPV model

In this section, we abuse the notation such that the LPV and LTI systems are represented in the same way. Readers are referred to section 3.2 to have a better understanding on how to derive the model for their use case.
The continuous time state-space realizations of output currents ($^{\Delta}$), circulating currents ($^{\Sigma}$) and zero-sequence current (^{z}) yield

$$\dot{x}^{\Delta}_{dq} = \mathbf{A}^{\Delta} x^{\Delta}_{dq} + \mathbf{B}^{\Delta} u^{\Delta}_{dq} + \mathbf{E}^{\Delta} w_{dq}, \tag{7a}$$
$$\dot{x}^{\Sigma}_{dq} = \mathbf{A}^{\Sigma} x^{\Sigma}_{dq} + \mathbf{B}^{\Sigma} u^{\Sigma}_{dq}, \tag{7b}$$
$$\dot{x}^{\Sigma}_{z} = \mathbf{A}^{z} x^{\Sigma}_{z} + \mathbf{B}^{z} u^{\Sigma}_{z} + \mathbf{E}^{z} w_{z}, \tag{7c}$$

respectively. Following Remark 3.3, the function matrices are defined as

$$\begin{array}{lll} \mathbf{A}^{\Delta} = A^{\Delta}_0 + \Delta A^{\Delta}, & \mathbf{B}^{\Delta} = B^{\Delta}_0 + \Delta B^{\Delta}, \text{and} & \mathbf{E}^{\Delta} = E^{\Delta}_0 + \Delta E^{\Delta}, \\ \mathbf{A}^{\Sigma} = A^{\Sigma}_0 + \Delta A^{\Sigma}, & \mathbf{B}^{\Sigma} = B^{\Sigma}_0 + \Delta B^{\Sigma}, & \\ \mathbf{A}^{z} = A^{z}_0 + \Delta A^{z}, & \mathbf{B}^{z} = B^{z}_0 + \Delta B^{z}, \text{and} & \mathbf{E}^{z} = E^{z}_0 + \Delta E^{z}, \end{array} \tag{8}$$

with the nominal dynamics as

$$\begin{aligned} A^{\Delta}_0 &= \begin{bmatrix} -\frac{R^0_{eq}}{L^0_{eq}} & -\omega \\ \omega & -\frac{R^0_{eq}}{L^0_{eq}} \end{bmatrix}, \quad B^{\Delta}_0 = \begin{bmatrix} \frac{1}{L^0_{eq}} & 0 \\ 0 & \frac{1}{L^0_{eq}} \end{bmatrix}, \quad E^{\Delta}_0 = \begin{bmatrix} \frac{1}{L^0_{eq}} & 0 \\ 0 & \frac{1}{L^0_{eq}} \end{bmatrix} \\ A^{\Sigma}_0 &= \begin{bmatrix} -\frac{R_{m0}}{L_{m0}} & 2\omega \\ -2\omega & -\frac{R_{m0}}{L_{m0}} \end{bmatrix}, \quad B^{\Sigma}_0 = \begin{bmatrix} -\frac{1}{L_{m0}} & 0 \\ 0 & -\frac{1}{L_{m0}} \end{bmatrix} \\ A^{z}_0 &= -\frac{R_{m0}}{L_{m0}}, \qquad B^{z}_0 = -\frac{1}{L_{m0}}, \qquad E^{z}_0 = \frac{1}{L_{m0}}. \end{aligned} \tag{9}$$

The uncertain dynamics are represented by the function matrices with a scheduling vector $\rho \in \mathbb{P}$ as in Remark 3.3. Note that there exist a deterministic method to derive $\mathbb{P}$ if the range of the uncertain values are known but that will be added in the next version of this documentation.

Remark 3.4. *The uncertain coefficients represented by the scheduling vector ρ can be contained in convex sets. Therefore, we define a set of convex sets that constraints the uncertain dynamics such that*

$$\begin{array}{lll} \Delta A^{\Delta} \in \mathbb{P}_{A^{\Delta}}, & \Delta B^{\Delta} \in \mathbb{P}_{B^{\Delta}}, & \Delta E^{\Delta} \in \mathbb{P}_{E^{\Delta}}, \\ \Delta A^{\Sigma} \in \mathbb{P}_{A^{\Sigma}}, & \Delta B^{\Sigma} \in \mathbb{P}_{B^{\Sigma}}, & \\ \Delta A^{z} \in \mathbb{P}_{A^{z}}, & \Delta B^{z} \in \mathbb{P}_{B^{z}}, & \Delta E^{z} \in \mathbb{P}_{E^{z}}, \end{array} \tag{10}$$

where $\mathbb{P}_{A^{\Delta}}, ..., \mathbb{P}_{E^{\Delta}}, ..., \mathbb{P}_{B^{\Sigma}} \subset \mathbb{R}^{2\times 2}$ and $\mathbb{P}_{A^{z}} ... \mathbb{P}_{E^{z}} \subset \mathbb{R}$.

3.2.2 Discrete time LTI and LPV model

Considering (7), we can obtain a discrete-time system using method such as 'zero-order-hold' (ZOH), backward and forward Euler, and Tustin. The discretizations are implemented using MATLAB function `c2d(...)`. Readers are referred to [5] for information about how to derive this equations.

3.2.3 Augmented model with integral action

To include integral action on the controller synthesis, we can derived an augmented systems. Let us consider a generic discrete-time state-space realization, i.e.,

$$\begin{aligned} x(k+1) &= Ax(k) + Bu(k) + Ew(k), \\ y(k) &= Cx(k), \end{aligned} \tag{11}$$

the augmented model yields

$$\begin{aligned} \begin{bmatrix} \delta x(k+1) \\ y(k+1) \end{bmatrix} &= \begin{bmatrix} A & 0 \\ CA & I \end{bmatrix} \begin{bmatrix} \delta x(k) \\ y(k) \end{bmatrix} + \begin{bmatrix} B \\ CB \end{bmatrix} \delta u + \begin{bmatrix} E \\ CE \end{bmatrix} \delta w \\ \vec{y}(k) &= \begin{bmatrix} 0 & I \end{bmatrix} \begin{bmatrix} \delta x(k) \\ y(k) \end{bmatrix} \end{aligned} \tag{12}$$

where the operator $\delta\cdot$ represents the discrete derivative such that $\delta x = x(k) - x(k-1)$, $\delta u = u(k) - u(k-1)$, $\delta w = w(k) - w(k-1)$. The new augmented state is defined as

$$\vec{x} := \begin{bmatrix} \delta x & y \end{bmatrix}^\top. \tag{13}$$

The above mentioned methodology can be applied to discrete-time representation of the LTI and LPV systems presented in Section 3.2.1.

3.3 MMC Control Problem

We consider the control problem of tracking piece-wise reference of the MMC currents.

Problem 3.1 (Classic Formulation)**.** *Given piece-wise references for the output current, circulating current or zero-sequences, such that*

$$r^{\Delta}(k+1) = r^{\Delta}(k), \quad \textit{with} \quad r^{\Delta}(k) = \begin{bmatrix} i^{\Delta}_{d_ref} & i^{\Delta}_{q_ref} \end{bmatrix}, \tag{14a}$$

$$r^{\Sigma}(k+1) = r^{\Sigma}(k), \quad \textit{with} \quad r^{\Sigma}(k) = \begin{bmatrix} i^{\Sigma}_{d_ref} & i^{\Sigma}_{q_ref} \end{bmatrix}, \tag{14b}$$

$$r^{z}(k+1) = r^{z}(k), \quad \textit{with} \quad r^{z}(k) = \begin{bmatrix} i^{\Sigma}_{z_ref} \end{bmatrix}, \tag{14c}$$

respectively. The MMC current control problem consists on finding u^{Δ}_{dq}, u^{Σ}_{dq} or u^{Σ}_{z} such that

$$\lim_{k\to\infty} \|x^{\Delta}_{dq}(k) - r^{\Delta}(k)\| = 0, \tag{15a}$$

$$\lim_{k\to\infty} \|x^{\Sigma}_{dq}(k) - r^{\Sigma}(k)\| = 0, \tag{15b}$$

$$\lim_{k\to\infty} \|x^{\Sigma}_{z}(k) - r^{z}(k)\| = 0, \tag{15c}$$

$$\tag{15d}$$

respectively.

This control problem is extended to consider constraint and modeling mismatches, which represent more realistic case of study for real-life applications. For example, in the case of the output current, the extended control problems yield

Problem 3.2 (Strict Formulation)**.** *Given piece-wise references for the output current,*

$$r^{\Delta}(k+1) = r^{\Delta}(k), \qquad with \quad r^{\Delta}(k) = \begin{bmatrix} i^{\Delta}_{d_ref} & i^{\Delta}_{q_ref} \end{bmatrix}, \tag{16}$$

The OCC problem consists on finding a constrained and robust u^{Δ}_{dq}, such that

$$\lim_{k\to\infty} \|x^{\Delta}_{dq}(k) - r^{\Delta}(k)\| = 0, \tag{17a}$$

$$s.t. \quad x^{\Delta}_{dq}(k) \in \mathbb{X} \subseteq \mathbb{R}^2 \quad \forall k \geq 0, \tag{17b}$$

$$u^{\Delta}_{dq}(k) \in \mathbb{U} \subseteq \mathbb{R}^2 \quad \forall k \geq 0, \tag{17c}$$

$$\tag{17d}$$

for all $\Delta A^{\Delta} \in \mathbb{P}_{A^{\Delta}}, \ldots, \Delta E^{\Delta} \in \mathbb{P}_{E^{\Delta}}$.

3.4 MPC Problem Formulation

The presented MPC solves at time k, the following finite-horizon optimal control problem (OCP)

$$\min_{U_k} J(x(k), U_k), \tag{18a}$$

$$\text{s.t. } x_{i+1|k} = Ax_{i|k} + Bu_{i|k} + Ew_{i|k}, \quad i = \{0, \ldots, N-1\} \tag{18b}$$

$$x_{i|k} \in \mathbb{X}, \quad i = \{1, \ldots, N\} \tag{18c}$$

$$u_{i|k} \in \mathbb{U}, \quad i = \{0, \ldots, N-1\} \tag{18d}$$

$$x_{0|k} = x(k) \tag{18e}$$

$$x_{N|k} \in \mathcal{X}_T \subseteq \mathbb{X} \tag{18f}$$

with $J((x(k), U_k))$ being a positive cost function, such that

$$J : \mathbb{R}^n \times \mathbb{R}^{Nm} \to \mathbb{R}_{\geq 0}, \quad \forall\, x(k) \in \mathbb{R}^n, U_k \in \mathbb{R}^{Nm} \tag{19}$$

where N is the prediction horizon, i.e., amount of time instants predicted by the model; $U_k = [u^{\top}_{0|k} \ldots u^{\top}_{N-1|k}]^{\top} \in \mathbb{R}^{Nm}$ is the predicted input vector and (x^s, u^s) are the desired set-points. Equation (18b) defines the prediction model, which is assumed controllable. The state and control input constraints formulated in (18c) and (18d), are defined by the sets

$$\mathbb{X} = \{x \in \mathbb{R}^n : x_{\min} \leq x \leq x_{\max}\}$$

and

$$\mathbb{U} = \{u \in \mathbb{R}^m : u_{\min} \leq u \leq u_{\max}\},$$

respectively. Condition (18f) describes the terminal constraints where $\mathcal{X}_T \subseteq \mathbb{X}$ is a terminal set that should satisfy the standard conditions for stability and recursive feasibility.

Depending of the problem different cost function can be designed. In the case of linear MPC, convex quadratic cost function are preferred. The most common ar the following:

- Classic stabilizing cost function, i.e., stability of the origin:

$$J(x(k), U_k) = \left\|x_{N|k}\right\|^2_P + \sum_{i=0}^{N-1} \left\|x_{i|k}\right\|^2_Q + \left\|u_{i|k}\right\|^2_R, \tag{20}$$

- Classic tracking cost function, i.e., tracking equilibrium points (x^s, u^s):

$$J(x(k), U_k) = \left\|x_{N|k} - x^s\right\|^2_P + \sum_{i=0}^{N-1} \left\|x_{i|k} - x^s\right\|^2_Q + \left\|u_{i|k} - u^s\right\|^2_R, \tag{21}$$

  where the equilibrium points (x^s, u^s) describe the steady-state of the reference such that $x^s = Ax^s + Bu^s + Ew(k)$ and $r = Cx^s$.

- Input-rate-penalized tracking cost function, i.e., tracking equilibrium points (x^s, u^s):

$$J(x(k), U_k) = \left\|x_{N|k} - x^s\right\|_P^2 + \sum_{i=0}^{N-1} \left\|x_{i|k} - x^s\right\|_Q^2 + \left\|u_{i|k} - u_{i-1|k}\right\|_R^2, \tag{22}$$

  where for $i = 0$, $u_{i-1|k} = u(k-1)$

- Classic output tracking cost function, i.e., tracking output references (y^r, u^s):

$$J(x(k), U_k) = \left\|y_{N|k} - r\right\|_P^2 + \sum_{i=0}^{N-1} \left\|y_{i|k} - r\right\|_Q^2 + \left\|u_{i|k} - u^s\right\|_R^2, \tag{23}$$

  where the output can be derived from the state measurements such that $y_{i|k} = Cx_{i|k}$, and y^r is the reference to be tracked such that $r = Cx^s$.

Remark 3.5 (Positive cost function)**.** *To ensure that all $J()$ are positive cost functions, we assume that weighting matrices $Q, P \in \mathbb{R}^{n\times n}$, $R \in \mathbb{R}^{m\times m}$ satisfy the conditions $Q = Q^\top \succeq 0, P = P^\top \succeq 0$, $R = R^\top \succ 0$.*

3.4.1 QP Programming of MPC controller

If the assumption in Remark 3.5 holds the MPC problem can be expressed as solvable QP problem. Then, given the current state $x(k)$, the finite horizon optimal control problem (18) can be reformulated as the following quadratic program (QP) with respect to U_k

$$\min_{U_k} \quad \frac{1}{2} U_k^\top G U_k + U_k F(x(k)) \tag{24a}$$

$$\text{s.t.} \quad LU_k \leq c + Wx(k). \tag{24b}$$

In (24b), $L \in \mathbb{R}^{q\times Nm}, W \in \mathbb{R}^{q\times n}$, and $c \in \mathbb{R}^q$ define the q constraints of the problem, while the cost function is defined by

$$G = 2\left(\Psi + \Gamma^\top \Omega \Gamma\right) \text{ and} \tag{25a}$$

$$F(x(k)) = 2\left[\Gamma^\top \Omega \left(\Phi x(k) - \mathbf{x}^s\right) - \Psi \mathbf{u}^s\right], \tag{25b}$$

where $(\mathbf{x}^s, \mathbf{u}^s)$ are stacked vectors containing the desired set-points for the complete prediction horizon. The matrices Γ and Φ are defined as

$$\Gamma = \begin{bmatrix} B & 0 & \dots & 0 \\ A\hat{B} & B & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ A^{N-1}B & A^{N-2}B & \dots & B \end{bmatrix}, \Phi = \begin{bmatrix} A \\ A^2 \\ \vdots \\ A^N \end{bmatrix} \tag{26}$$

and $\Omega \in \mathbb{R}^{Nn\times Nn}, \Psi \in \mathbb{R}^{Nm\times Nm}$ as

$$\Omega = \begin{bmatrix} Q & 0 & \dots & 0 \\ \vdots & \ddots & \vdots & \vdots \\ 0 & \dots & Q & 0 \\ 0 & \dots & 0 & P \end{bmatrix}, \Psi = \begin{bmatrix} R & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & R \end{bmatrix}. \tag{27}$$

We denote by U_k^* the optimal solution of problem (24). In the case, the first element of U_k^* is applied to the system while the rest is discarded, i.e., receding horizon principle. Given U_k^*, the

i^{th} constraint is called active at the optimum if $[LU_k^*]_i = [c+ Wx(k)]_i, i \in \mathbb{Z}_{[1,q]}$. If no constraint is active, the optimizer of (24) is the unconstrained solution

$$U_k^* = \begin{bmatrix} u_{0|k}^* \\ \vdots \\ u_{N-1|k}^* \end{bmatrix} = -G^{-1}F(x(k)). \tag{28}$$

Remark 3.6 (Nominal Prediction Model)**.** *The prediction model cannot account for the uncertain dynamics as by definition they are unknown. Therefore, we consider prediction model that considers only the nominal dynamics (rated values), such that*

$$A = A_0, \quad B = B_0. \tag{29}$$

Remark 3.7 (Augmented Model Use)**.** *For the **MMC** augmented state-space representation, we can disregard $E_0\delta w$, because it is assumed constant pice-wise w, which implies that $\delta w = 0$. Furthermore, the steady-state equilibrium point (x^s, u^s) corresponding to r have always the following structure*

$$x^s := \begin{bmatrix} 0_{1\times 5} & r^\top \end{bmatrix}^\top u^s := 0_{5\times 1}. \tag{30}$$

Remark 3.8 (Input-rate-penalized cost function formulation)**.** *For the case when the input-rate is penalized as in (22), the definition of Ψ and $\mathbf{u}^s$ change such that*

$$\Psi = [\star]^\top \begin{bmatrix} R & 0 & \dots & \dots & 0 \\ 0 & R & \dots & \dots & 0 \\ 0 & 0 & R & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & R \end{bmatrix} \begin{bmatrix} I_m & 0 & \dots & \dots & 0 \\ -I_m & I_m & \dots & \dots & 0 \\ 0 & -I_m & I_m & \dots & 0 \\ \vdots & \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & \dots & -I_m & I_m \end{bmatrix} \text{ and } \mathbf{u}^s = \begin{bmatrix} u(k-1) \\ 0 \\ \vdots \\ 0 \end{bmatrix}. \tag{31}$$

3.5 State feedback controller synthesis

Considering a discrete-time LPV[1]state-space realization, i.e.,

$$x(k+1) = A(\rho)x(k) + B(\rho)u(k), \tag{32}$$

then, state feedback controllers with different properties can be obtained solving a set of linear matrix inequalities (LMIs).

3.5.1 Linear Robust/Quadratic Design

The following LMI find a state feedback controller ($u = Kx$) that offers robust and quadratic stability for all possible $\rho \in \mathbb{P}$, i.e., modeling mismatches.

$$\begin{bmatrix} Z & (A(v^j)Z + B(v^j)Y)' \\ (A(v^j)Z + B(v^j)Y) & Z \end{bmatrix} \succ 0, \quad \forall j \in \{1, ..., p\} \tag{33}$$

where $K = YZ^{-1}$, p is number of vertices of $\mathbb{P}$, v^j are the vertices of $\mathbb{P}$ and $\mathbb{P}$ is convex set containing the scheduling variables ρ.

[1]Only LPV models with affine embedding are considered.

3.5.2 Linear Constrained Robust Design

The following LMIs find a state feedback controller with a domain of attraction in which it guarantees state and input constraint satisfaction, i.e., $\mathbb{S} := \{x \in \mathbb{R}^n : x^\top Z x \leq 1\} \subseteq \mathbb{X}$.

$$\begin{bmatrix} Z & (A(v^j)Z + B(v^j)Y)' \\ (A(v^j)Z + B(v^j)Y) & Z \end{bmatrix} \succ 0, \quad \forall j \in \{1, ..., p\}, \tag{34a}$$

$$\begin{bmatrix} Z & (Zg_{t_x})^\top \\ (Zg_{t_x}) & 1 \end{bmatrix} \succcurlyeq 0, \quad \forall t_x \in \{1, \ldots, s\}, \tag{34b}$$

$$\begin{bmatrix} Z & (Yh_{t_u})^\top \\ (Yh_{t_u}) & 1 \end{bmatrix} \succcurlyeq 0, \quad \forall t_u \in \{1, \ldots, l\}, \tag{34c}$$

where $K = YZ^{-1}$, g_{t_x} and h_{t_u} are normalized vectors corresponding to the error constraints from $\mathbb{X}$ and $\mathbb{U}$, respectively, p is number of vertices of $\mathbb{P}$, v^j are the vertices of $\mathbb{P}$ and $\mathbb{P}$ is convex set containing the scheduling variables ρ.

3.5.3 Linear Optimal Robust Design [1, 2]

3.6 MPC Tuning via Controller Matching

The following optimization problem finds Q, R, P and $\mathcal{X}_T$, that guarantees that MPC closed-loop performance resembles the properties of a pre-computed controller, e.g., robustness, transient response or stability [6]. Given a desired state feedback gain K, MPC tuning via controller matching is solved by

$$J = \min_{P,Q,R,\varepsilon} \ \varepsilon \tag{35a}$$

$$\text{s.t. } \left\| \left(\left(\Psi + \Gamma^\top \Omega \Gamma \right) \mathcal{K} + \Gamma^\top \Omega \Phi \right) \cdot \xi^j \right\| \leq \varepsilon, \qquad \forall j = \{1, \ldots, p\}, \tag{35b}$$

$$(A + BK)^\top P (A + BK) + K^\top R K + Q - P \preccurlyeq 0, \tag{35c}$$

$$Q \succ 0, \ P \succ 0, \ R \succeq \sigma I, \tag{35d}$$

$$\varepsilon \geq 0, \ \sigma > 0. \tag{35e}$$

where ξ^j are the vertices of the constraint set $\mathbb{X}$ and

$$\mathcal{K} = \begin{bmatrix} K_{db} \\ K_{db}(A + BK_{db}) \\ \vdots \\ K_{db}(A + BK_{db})^{N-1} \end{bmatrix}. \tag{36}$$

4 Controller Architecture

5 Getting Started

5.1 Basic Workflow

The typical workflow when using the toolbox is as follows

1. Define system dynamics, i.e, (A, B, C, E) matrices.
2. Define system constraints including for the scheduling variables, i.e., $(\mathbb{X}, \mathbb{U})$ sets.
3. Design matching controller:
   (a) Define the closed-loop performance, e.g., classic, constrained, robust.
   (b) Compute the state feedback gain K as in (33), or in (34).

4. Set MPC hyperparameters:
   (a) Define N, i.e., the prediction horizon.
   (b) Compute (Q, R, P) via matching controller technique as in (35).
   (c) Compute terminal constraints $\mathcal{X}_T$.
5. Create an MPC-OCP structure:
   (a) Given (Q, R, P), compute the compact weighting matrices (Ψ, Ω) as in (27)
   (b) Given (A, B, N), compute the compact prediction model matrices (Φ, Γ) as in (26)
   (c) Given $(\mathbb{X}, \mathbb{U}, \Phi, \Gamma)$, compute the constraints matrix and vector (L, W, c) as in (24b)
   (d) Give $(\Psi, \Omega, \Gamma, \Phi)$, compute the quadratic program (QP) matrices (G, F) as in (25)
6. Solve the QP problem as in (24) using Hildreth's algorithm.
7. Analyze and visualize results.
8. Deploy controller in Simulink, CBuilder or C++ MEX.

Readers are referred to Figure 19 in the appendix.

5.1.1 Basic workflow for defining system dynamics

Figure 1 shows the basic workflow for defining the system dynamics. This toolbox considers any system that can be represented by a state-space representation. Since it is tailored for MMC applications, it offers functions that can directly generate an LTI model of the MMC. These functions can generate both per-unit (`getPlantModel`) and per-magnitude models (`getPlantModelPU`). The constraints must be define by matrix containing the maximum and minimum values, then `Xbound`$\leftarrow \mathbb{X}$ and `Ubound`$\leftarrow \mathbb{U}$. In the case of scheduling variables should be defined by a matrix containing all the vertices of the constraint set $\mathbb{P}$.

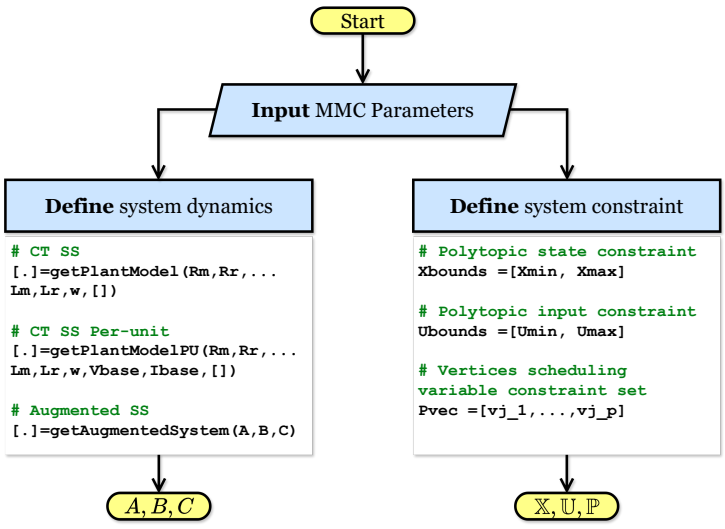


Figure 1: Flow diagram for defining the systems dynamics and constraints

5.1.2 Basic workflow for designing matching controller and MPC tuning

Figure 2 shows the basic workflow for defining the matching controller, which determines the transient dynamics of the MPC in closed loop when automatic tuning is used. The toolbox relies on a single function for this process, i.e., `getKRobust(·)`. However, depending on the desired closed-loop performance, the user can synthesize different controllers with distinct properties. By changing the variable `method`, the user can choose between solving (33), (34) or a classic LMI.

Readers are referred to Example 8.1 for observing how to compute different K and how they compare in performance.

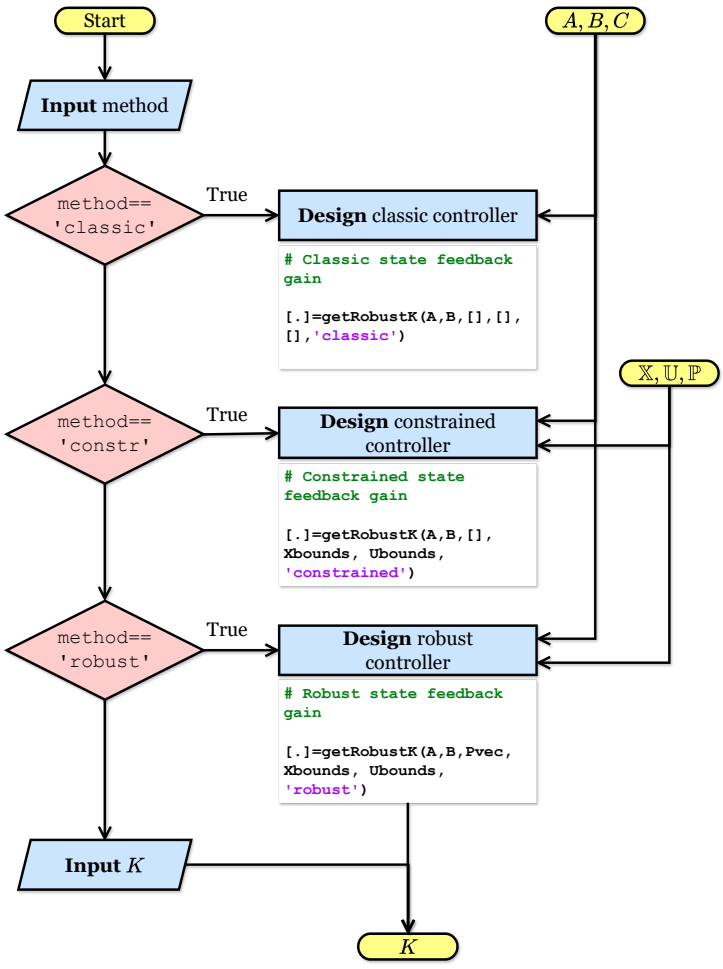


Figure 2: Flow diagram for defining the matching controller

Remark 5.1. *The classic method solves an LMI based on the Lyapunov inequality, providing quadratic stability guarantees but no robustness.*

Remark 5.2. *Function* `getKRobust(·, 'robust')`*, which corresponds to the computation of robust K, considers only* ***symmetric scheduling variable sets****, i.e.,*

$$\mathbb{P} := \{\rho \in \mathbb{R}^{ab\times 1} : \rho_{min} \leq \rho \leq \rho_{max} \quad with \quad \rho_{max} = -\rho_{min}\}.$$

As a consequence, the `Pvec` $= \rho_{max}$*, i.e., the a vector of the maximum values that each ρ_i can take instead of the vertices of $\mathbb{P}$. To considers different types of sets modifications can be done to* `getKRobust(·)`*, such modifications will be included in new version of the manual.*
Rahul's in the current version, this function only handles 2×2 systems. I will fix it later.:)

Figure 3 shows the proposed workflow for tuning the MPC. The toolbox includes a function, `matchingControllerDesign(·)`, which solves (35a) and computes the optimal Q, R, and P matrices for a given prediction horizon N, system dynamics, and matching controller K. Note that K can be any matrix with compatible dimensions; this tuning method is not limited to the controller synthesized by `getKRobust(·)`.

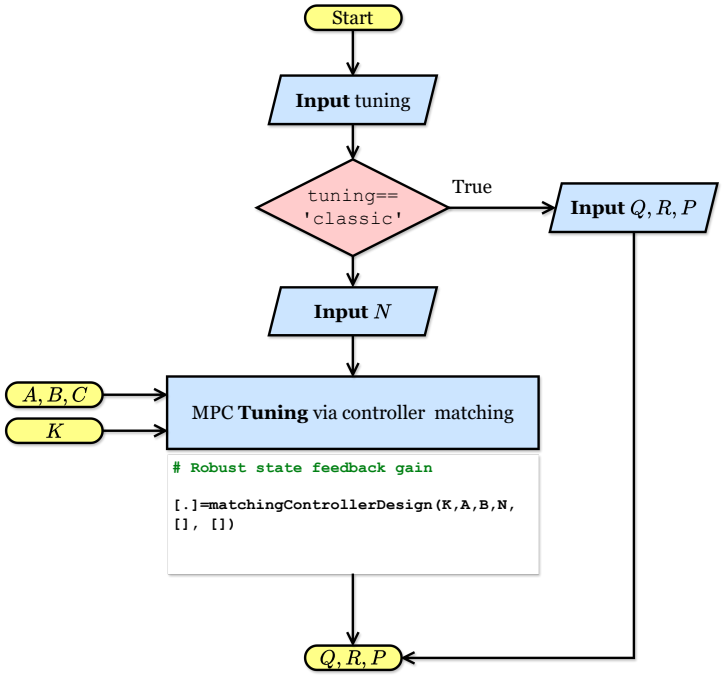


Figure 3: Flow diagram for MPC tuning via controller matching

5.1.3 Basic workflow for creating MPC structure

Figure 4 shows the workflow for creating the MPC-OCP structure used to solve the QP problem with Hildreth's algorithm. It is not necessary to create a `struct`-type variable to solve the QP problem. The only necessary and sufficient condition is to compute the components presented in Figure 4.

Remark 5.3. *The constraints can be added individually. By performing column concatenation, different types of constraints can be combined. In this toolbox, there are 6 functions available to compute the constraint matrices:*

- *`getWLc(·)` refers to the computation of the state and input constraint matrices as in (24b). Here, the state and input constraint cannot be decoupled. Readers are referred to [7] for more information on how these matrices are computed.*
- *`getMb(·, method)` refers to the computation of the input constraint matrices only such that (24b) becomes $MU_k \leq b$. Note that if we consider an augmented system the decision variable are ΔU_k. In this case, to penalize the true control input of the real system, i.e., $u_{min} \leq u \leq u_{max}$, used `method ='delta'`.*
- *`getAXbx(·)` refers to the computation of the state constraint matrices such that (24b) becomes $MU_k \leq b$, such that*

$$MU_k \leq b, \quad with \quad M \in \mathbb{R}^{Nn \times Nm},\ b = \left[\boldsymbol{xmin}^\top, \boldsymbol{xmax}^\top, ..., \boldsymbol{xmin}^\top, \boldsymbol{xmax}^\top\right]^\top. \tag{37}$$

  See Section for more information.

- *`getAYby(·)` refers to the computation of the output constraint matrices only such that (24b) becomes $MU_k \leq b$, such that*

$$MU_k \leq b, \quad with \quad M \in \mathbb{R}^{Np \times Nm},\ b = \left[\boldsymbol{ymin}^\top, \boldsymbol{ymax}^\top, ..., \boldsymbol{ymin}^\top, \boldsymbol{ymax}^\top\right]^\top. \tag{38}$$

  See Section for more information.

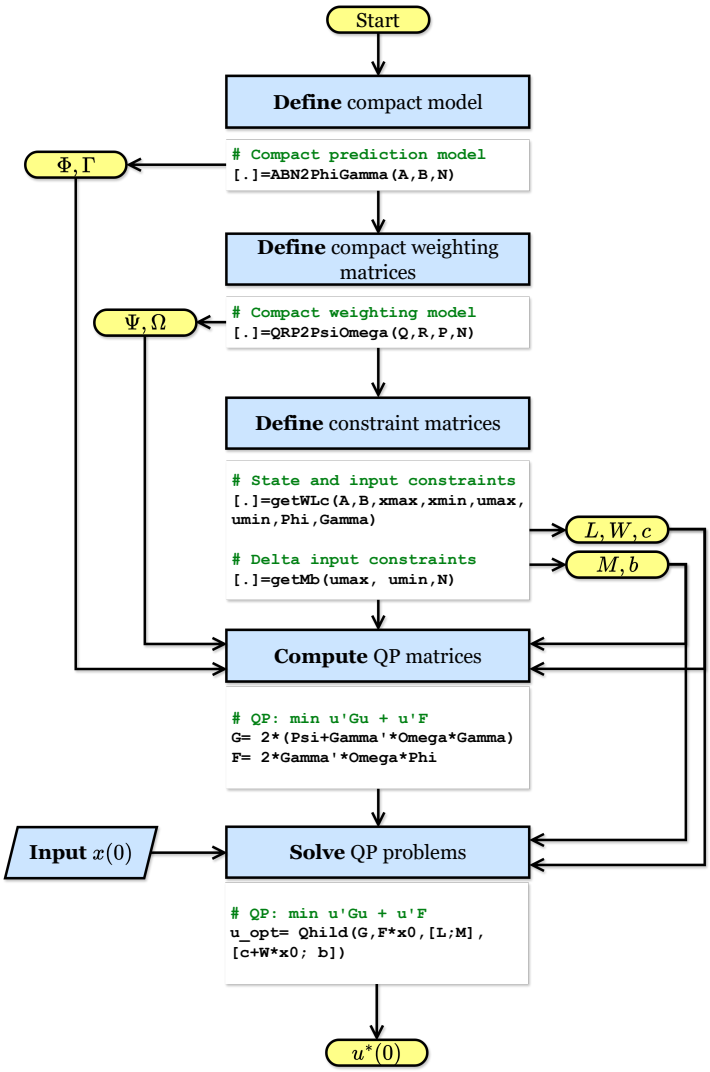


Figure 4: Flow diagram for creating an MPC-OPC structure

- *`getAUbu(·)` refers to the computation of the output constraint matrices only such that (24b) becomes $MU_k \leq b$, such that*

$$MU_k \leq b, \quad with \quad M \in \mathbb{R}^{Np \times Nm},\ b = \left[\boldsymbol{umin}^\top, \boldsymbol{umax}^\top, ..., \boldsymbol{umin}^\top, \boldsymbol{umax}^\top\right]^\top. \tag{39}$$

See Section for more information.

- *`getdAUbdu(·)` refers to the computation of the input change rate constraint matrices only such that (24b) becomes $MU_k \leq b$, such that*

$$MU_k \leq b, \quad with \quad M \in \mathbb{R}^{Np \times Nm},\ b = \left[\boldsymbol{iumin}^\top, \boldsymbol{iumax}^\top, ..., \boldsymbol{iumin}^\top, \boldsymbol{iumax}^\top\right]^\top. \tag{40}$$

See Section for more information.

- *`getAiUbiu(·)` refers to the computation of the input integral constraint matrices only such that (24b) becomes $MU_k \leq b$, such that*

$$MU_k \leq b, \quad with \quad M \in \mathbb{R}^{Np \times Nm},\ b = \left[\boldsymbol{dumin}^\top, \boldsymbol{dumax}^\top, ..., \boldsymbol{dumin}^\top, \boldsymbol{dumax}^\top\right]^\top. \tag{41}$$

See Section for more information.

This toolbox solves the QP problem using `QPHild(·)` or `QPHildClassic(·)`. Both are implementation of the Hildreth's algorithm in [8]. Furthermore, similar functions have been implemented in [5]. There are not significant differences between `QPHild(·)` or `QPHildClassic(·)`, only that `QPHildClassic(·)` has as input the inverse of G, i.e., G^{-1}.

The following excerpt of code shows how to create the MPC-OCP structure and solve the QP problem.

Note: The system considered is based on the augmented model, therefore we used `getMB(Mb,umax,umin,N,'delta')`. In this case, the bounds for the Δu and u are the same.

```
% Previous part of the code
...
xmax = Xbound(:,2);
xmin = Xbound(:,1);

umax = Ubound(:,2);
umin = Ubound(:,1);

nx =  length(xmax);
nu =  length(umax);

% Compact prediction model
[ Phi, Gamma ] = ABN2PhiGamma(A,B,N);
[ Psi, Omega ] = QRPN2PsiOmega( Q,R,P,N );

% Constraints definition
[ W, L, c] = getWLc( A,B,xmax,xmin,umax,umin,Gamma,Phi );   % State and delta input
    constraints
[Mcal,b] = getMb(umax,umin,N,'delta');                      % Input constraints

% Compounding constraints
M = [L;Mcal];

G = 2*(Psi+Gamma'*Omega*Gamma);
F = 2*Gamma'*Omega*Phi;
```

```
25
26 invG =  inv(G);
27
28
29 % Initial conditions
30 x0 = ... ; % [n x 1] Initial conditions at time  k
31 u0 = ... ; % [m x 1] Initial control input at time  k
32
33 % Contraint vector at time k
34 bpast = [ones(nu,1).*u0; -ones(nu,1).*u0; zeros(2*nu*(N-1),1)];
35 gamma = [c+W*z_k;b-bpast];
36
37 % Solving QP problem
38 u_opt = QPHildClassic(invG, F*x0, M, gamma)     % This exactly Rohan's function but I
    pre-compute the inverse
```

5.1.4 MPCClass Class

This toolbox defines an `MPCClas` class to create `MPC` objects. The `MPC` object is structure capable to solve the MPC-OCP problem too. This class offers a more compact manner to solve the MPC-OCP problem. Readers are referred to Section for more information.

6 Toolbox Architecture

6.1 Folder Structure

A typical folder layout is:

```
rmpc_toolbox/
    +-- lib/
    +-- class/
    +-- examp/
    +-- src/
    +-- doc/
```

- `lib/`: Contains all the functions
- `class/`: Contains all the class definition.
- `lib/src`: Contains MATLAB/Simulink implementation.
- `examples/`: Example scripts and demo models.
- `docs/`: Documentation source (this file).

7 MATLAB Function Reference

This section documents the main public functions of the toolbox. We divide the section based on the workflow.

7.1 System dynamics definition

7.1.1 `getPlantModel`

Syntax

```
[Ac, Bc, Cc] = getPlantModel(Rarm, Rr, Larm, Lr, w, type)
```

Description `getPlantModel(type, Rarm, Rr, Larm, Lr, w)` generates the A,B,C matrices corresponding to continous-time state-space realization of the MMC model, i.e., $x(k+1) = Ax(k) + Bu(k)$.

Inputs

- `Rarm` in [Ohm], i.e., the arm resistance,
- `Rr` in [Ohm], i.e., the grid resistance,
- `Larm` in [H], i.e., the arm inductance,
- `Lr` in [H], i.e., the grid inductance,
- `type` in [string], i.e., the defines the type of model to be used, such as `'OCC'`, `'CCC'` and `'ZCC'` correspond to the OCC, CCC and ZCC, respectively.

Outputs `[Ac, Bc, Cc]` state-space realization matrices

7.1.2 `getPlantModelPU`

Syntax

```
[Ac, Bc, Cc] = getPlantModelPU(Rarm, Rr, Larm, Lr, w, Ibase, Vbase, type)
```

Description `getPlantModelPU(type, Rarm, Rr, Larm, Lr, w, Ibase,Vbase,type)` generates the A,B,C matrices corresponding to continous-time state-space realization of the MMC model, i.e., $x(k+1) = Ax(k) + Bu(k)$, where the magnitudes are **per unit**.

Inputs

- `Rarm` in [Ohm], i.e., the arm resistance,
- `Rr` in [Ohm], i.e., the grid resistance,
- `Larm` in [H], i.e., the arm inductance,
- `Lr` in [H], i.e., the grid inductance,
- `Ibase` in [A], i.e., the base current in the AC-side,
- `Vbase` in [V], i.e., the base voltage in the AC-side.
- `type` in [string], i.e., the defines the type of model to be used, such as `'OCC'`, `'CCC'` and `'ZCC'` correspond to the OCC, CCC and ZCC, respectively.

Outputs `[Ac, Bc, Cc]` state-space realization matrices

7.1.3 `getAugmentedSystem`

Syntax

```
[Ag, Bg, Cg] = getAugmentedSystem(A, B, C)
```

Description `getAugmentedSystem(A, B, C)` generates the augmented state-space realization corresponding to discrete-time state-space models. The augmented system is implemented as described in Section 3.2.3.

Inputs

- `A` state matrix,
- `B` input matrix,
- `C` output matrix.

Outputs `[Ag, Bg, Cg]` state-space realization matrices of the augmented system.

7.2 Synthesis state feedback controller

7.2.1 `getSteadyState`

```
[xss,uss,erorPredict]=getSteadyState(A,B,C,D,E,y,d,xss,method)
```

Description `getSteadyState(A,B,C,D,E,y,d,xss,method)` computes the linear system's steady-state such that `xss = A*xss +B*uss+E*d` and `y = C*xss +D*uss`.

Inputs

- `[A,B,C,D,E]` state-space matrices,
- `y` output reference
- `d` disturbance
- `xss` state reference (steady-state)
- `method [string]` to select the method to compute the steady state, e.g., `'state'`, `'output'` and `'perturbed'`

Outputs

- `[xss,uss]` computed steady-state for the state and the control input
- `erorPredict` computation error

7.2.2 `getRobustK`

```
[K,P,diag]=getRobustK(A,B,C,pvec,xbound,ubound,methodF)
```

Description `getRobustK(A,B,C,pvec,xbound,ubound,methodF)` computes a robust state feedback gain for an LPV representation of an uncertain linear model, i.e., $x(k+1) = A(\rho)x + B(p)u + E(\rho)w$, such that $\|\lambda_{max}(A(\rho) + B(\rho)K)\| < 1$. This function is tailored for MMC model dynamics, including augmented system presented in Section 3.2.

Inputs

- `[A,B,C]` state-space matrices,
- `pvec` vector containing the maximum value each element in ρ.
- `xbound` $\left[x_{min}, x_{max}\right]$ matrix containing the maximum and minimum constraints of the states
- `ubound` $\left[u_{min}, u_{max}\right]$ matrix containing the maximum and minimum constraints of the inputs
- `methodF [string]` to select the method to compute the LMIs, e.g., `'robust'`, and `'constrained'`.

Outputs

- `K` computed state feedback gain
- `P` computed domain of attraction

7.3 MPC tuning via controller matching

7.3.1 `matchingControllerDesignRelax`

Syntax

```
[Q,R,P,J,diagnostic]=matchingControllerDesignRelax(K,A,B,N,vj,sigma)
```

Note: This function will be updated in future version for `matchingFnc`!

Description `matchingControllerDesignRelax(K,A,B,N,vj,sigma)` enerates the matrices (Q,R,P) for tuning the MPC. This method follows the solution proposed in [6], specifically solving [6, Lemma 3.1].

Inputs

- `K` state feedback gain matrix
- `A` state matrix
- `B` input matrix
- `C` output matrix
- `N` prediction horizon [scalar].
- `vj` contains the vector of the polytope P as in [6, (22)].
- `sigma` hyperparameter to ensure $R > 0$, then $\sigma \in (0, 1]$.

Outputs

- `Q` state weight matrix
- `R` input weight matrix
- `P` terminal cost matrix
- `J` value function of J as [6, (24a)]
- `diagnostic` [struct] returns the diagnostic from Yalmip solver

7.4 MPC Formulation

7.4.1 `ABN2PhiGamma`

Syntax

```
[ Phi, Gamma ] = ABN2PhiGamma(A,B,N)
```

Description `ABN2PhiGamma(A,B,N)` generates the prediction matrices `Phi` $= \Phi$, `Gamma` $= \Gamma$ corresponding to the prediction model of the linear MPC implementation as in [7]. The compact prediction model yields $Xk = \Phi x_0 + \Gamma Uk$, where $x_0 = x(k)$, $Xk = \{x(0), ..., x(N)\}$ and $\mathrm{U}k = \{u(0), ..., u(N)\}$.

Inputs

- `A` state matrix
- `B` input matrix
- `N` prediction horizon

Outputs `[ Phi, Gamma]` compact prediction model matrices.

7.4.2 `QRPN2PsiOmega`

Syntax

```
[ Psi, Omega ] = QRPN2PsiOmega( Q,R,P,N )
```

Description `QRPN2PsiOmega(Q,R,P,N)` generates the weighting matrices Psi, Omega corresponding to the quadratic cost function as in [7]. The weighting matrices `Psi`, `Omega` yields `Omega = diag(Q,...,Q,P)` and `Psi = diag(R,...,R)`.

Inputs

- `Q` state weighting matrix
- `P` terminal state weighting matrix (terminal cost)
- `R` input weighting matrix
- `N` prediction horizon

Outputs `[ Psi, Omega]` compact weighting matrices.

7.4.3 `getWLc`

Syntax

```
[ W, L, c] = getWLc( A, B, Xmax, Xmin, umax, umin, Gamma, Phi)
```

Description `getWLc( A, B, Xmax, Xmin, umax, umin, Gamma, Phi)` generates the matrices corresponding to the inequalities constraint of an QP problem, i.e., $MU_k \leq b$, where $M =$ `L` and $b =$ `c + W`x_0.

Inputs

- `A` state matrix
- `B` input matrix
- `[ Phi, Gamma]` compact prediction model matrices
- `[ Xmin, Xmax]` state constraints
- `[ umin, umax]` control input constraints

Outputs `[ W, L, c]` compact inequalities matrices.

7.4.4 `getMb`

Syntax

```
[M,b] = getMb(umax, umin, N, mehod)
```

Description `getMb(umax, umin, N, method)` generates the matrices corresponding to the inequalities constraint of an QP problem, i.e., $MU_k \leq b$.

Inputs

- `N` [scalar] prediction horizon
- `[ umin, umax]` $[m \times 1], [m \times 1]$ control input constraints
- `method` [string] to select how to constraints the inputs

Outputs `[M,b]` compact inequalities matrices.

7.4.5 `getAXbx`

Syntax

```
[Axu,b,Mx]=getAXbx(Gamma,Phi,N,xmax,xmin)
```

Description `getAXbx(Gamma,Phi,N,xmax,xmin)` generates the matrices corresponding to the state inequalities constraint of an QP problem, i.e., $MU_k \leq b$ with $M :=$ `Axu` and $b :=$ `Mx` $*$ `b`.

Inputs

- `N` [scalar] prediction horizon
- `[ Phi, Gamma]` compact prediction model matrices
- `[ xmin, xmax]` $[n \times 1], [n \times 1]$ control input constraints

Outputs `[Axu,b,Mx]` compact inequalities matrices.

7.4.6 `getAyby`

Syntax

```
[Ayu,b,My]=getAYby(Gamma,Phi,C,N,xmax,xmin)
```

Description `getAyby(Gamma,Phi,N,xmax,xmin)` generates the matrices corresponding to the output inequalities constraint of an QP problem, i.e., $MU_k \leq b$ with $M :=$ `Ayu` and $b :=$ `My` $*$ `b`.

Inputs

- `N` [scalar] prediction horizon
- `C` $[p \times n]$ state-to-output matrix
- `[ Phi, Gamma]` compact prediction model matrices
- `[ xmin, xmax]` $[n \times 1], [n \times 1]$ control input constraints

Outputs `[Ayu,b,My]` compact inequalities matrices.

7.4.7 `getAUbu`

Syntax

```
[Au,bu]=getAUbu(N,umax,umin)
```

Description `getAUbu(N,umax,umin)` generates the matrices corresponding to the input inequalities constraint of an QP problem, i.e., $MU_k \leq b$ with $M :=$ `Adu` and $b :=$ `bdu`.
Note: `bdu` needs to be modify such as `bdu(1) = bdu +upast`.

Inputs

- `N` [scalar] prediction horizon
- `[umin, umax]` $[m \times 1], [m \times 1]$ control input constraints

Outputs `[Au,bu]` compact inequalities matrices.

7.4.8 `getAdUbdu`

Syntax

```
[Adu,bdu]=getAdUbdu(N,umax,umin)
```

Description `getAdUbdu(N,umax,umin)` generates the matrices corresponding to the input change rate inequalities constraint of an QP problem, i.e., $MU_k \leq b$ with $M :=$ `Au` and $b :=$ `bu`.

Inputs

- `N` [scalar] prediction horizon
- `[umin, umax]` $[m \times 1], [m \times 1]$ control input change rate constraints

Outputs `[Adu,bdu]` compact inequalities matrices.

7.4.9 `getAiUbiu`

Syntax

```
[Aiu,biu]=getAiUbiu(N,umax,umin)
```

Description `getAiUbiu(N,umax,umin)` generates the matrices corresponding to the input integral inequalities constraint of an QP problem, i.e., $MU_k \leq b$ with $M :=$ `Aiu` and $b :=$ `biu`.

Inputs

- `N` [scalar] prediction horizon
- `[umin, umax]` $[m \times 1], [m \times 1]$ control input integral constraints

Outputs `[Aiu,biu]` compact inequalities matrices.

7.4.10 `QPHildClassic`

Syntax

```
eta = QPHildClassic(invOmega, f, M_cons, gamma, max_iter, tol)
```

Description `QPHildClassic(invOmega, f, M_cons, gamma, max_iter, tol)` solves Quadratic Programming using Hildreth's method, such that
Minimize: `0.5 * eta' * Omega * eta + f' * eta`,
Subject to: `M_cons * eta <= gamma`,
With: `Omega` = $\texttt{invOmega}^{-1}$.

Inputs

- `invOmega` quadratic term, i.e., $[n \times n]$ matrix
- `f` linear term, i.e., $[n \times 1]$ matrix
- `M_cons` inequality constraint matrix, i.e., $[m \times n]$ matrix
- `gamma` constraint vector, i.e., $[m \times 1]$ vector
- `max_iter` maximum number of iterations (hyperparameter)
- `tol` tolerance of the solver

Outputs `eta` optimal decision variable, i.e., the control input

8 Examples

8.1 Example: Matching controller computation

This example demonstrates how to use the function `getKRobust()` to compute different controllers and how their performance compares to each other and to the classic LQR controller.

Consider a generic discrete-time state-space realization, i.e.,

$$\begin{aligned} x(k+1) &= (A_0 + \Delta A)x(k) + Bu(k) \\ y(k) &= Cx(k) \end{aligned} \tag{42}$$

with

$$A_0 = \begin{bmatrix} 0.9746 & 0.0779 \\ -0.4675 & 0.5850 \end{bmatrix}, \quad B = \begin{bmatrix} 0.0042 & 0.0099 \\ 0.0779 & -0.0025 \end{bmatrix}, \quad C = [-1 \quad 1] \tag{43}$$

where the uncertainty is represented by ΔA, i.e.,

$$\Delta A \in \mathbb{R}^{n\times n} : \Delta A := \begin{bmatrix} \rho_1 & \rho_2 \\ \rho_3 & \rho_4 \end{bmatrix} \quad \text{with} \quad (\rho_2, \rho_4) \in [-0.4, 0.4], \quad \text{and} \quad (\rho_1, \rho_3) = 0. \tag{44}$$

Consider the following constraints

$$\mathbb{X} := \{x \in \mathbb{R}^2 : \|x\|_\infty \leq 1\} \quad \text{and} \quad \mathbb{U} := \{u \in \mathbb{R}^2 : \|u\|_\infty \leq 1\}. \tag{45}$$

The next excerpt of code shows how to compute various matching controllers for model (42).

```
% Example 1: Computation of different controllers using getKRobust()
% Author: Victor Daniel Reyes Dreke
% ------------------------------------------------------------------ %
% CT-SS Realization. Remark: Non-minimum phase system
Ac =  [0, 1; -6, -5];
Bc =  [0, 0.1; 1, 0];
Cc =  [-1, 1];

% DT-SS Realization
Ts =  0.1;                          % [s] Sampling time
sysd = c2d(ss(Ac,Bc,Cc,[]),Ts);     % ZOH Discretization

% SS DT Matrices
A = sysd.A; B = sysd.B; C = sysd.C;

% Polytopic state and input constraints
Xbound = [[1; 1], [-1; -1]];            % [Xmax, Xmin]
Ubound = [[1; 1], [-1; -1]];            % [Umax, Umin]
Pvec = [0, 0.4, 0, 0.4, zeros(1,4)];    % Pvec = [1 x nxm]

% Matching controllers design
% Case 1: LQR
Q = eye(2); R = 0.01*eye(2);
K_LQR = dlqr(A,B,Q,R);

% Case 2: Robust
[K_Robust,P_Robust]=getRobustK(A,B,[],Pvec,Xbound,Ubound,"robust");

% Case 3: Constrained
[K_Const,P_Const]=getRobustK(A,B,[],Pvec,Xbound,Ubound,"constrained");

% Case 4: Classic
[K_Class,P_Class]=getRobustK(A,B,[],Pvec,Xbound,Ubound,"classic");
```

To run the previous code, use:

Code

```
RobCMMC_Exmp_getKRboust
```

Running this code yields the following matching controllers:

$$\begin{aligned} \texttt{K_LQR} &= \begin{bmatrix} -0.5851 & -3.5392 \\ -7.0519 & -0.4677 \end{bmatrix}, \quad \texttt{K_Class} = \begin{bmatrix} 2.7505 & -7.6580 \\ -99.5016 & -4.5842 \end{bmatrix}, \\ \texttt{K_Robust} &= \begin{bmatrix} 2.7535 & -6.1484 \\ -99.4922 & 4.5210 \end{bmatrix}, \quad \texttt{K_Const} = \begin{bmatrix} 5.4127 & -3.5900 \\ -5.7448 & 2.0796 \end{bmatrix}. \end{aligned} \tag{46}$$

Observe that in `line 19`, we define `Pvec`. This variable does not follow the $\mathbb{P}$ presented in the mathematical background section, where $\mathbb{P}$ is defined by matrix containing all the vertices of the set. This `Pvec` contains the maximum value of each one of the scheduling variable vector, i.e., $\rho =: [\rho_1, ..., \rho_{n^3m}]$ with $n = 2$ and $m = 2$. Function `getKRobust()` compute the vertices of a symmetric set $\mathbb{P}$ corresponding to `Pvec`, such that

$$\texttt{Pvec = [0, 0.4, 0, 0.4, zeros(1,4)]} \implies |\rho| \leq \rho_{max} \tag{47}$$

with

$$\rho_{max} = \begin{bmatrix} 0 & 0.4 & 0 & 0.4 & 0 & 0 & 0 & 0 \end{bmatrix}. \tag{48}$$

Remark 8.1. *In futures version of the this toolbox, we expect to increment the flexibility of the `getKRobust()` function to consider other types of sets.*

Figure 5 shows the simulations of **the nominal** (42) in closed-loop with the different matching controllers in (46). The initial conditions for all these simulation are the same, i.e., $x(0) = [0.5 \quad -0.5]^\top$. From this example, the following conclusions can be highlighted:

- The classic matching controller, i.e., `method = 'classic'`, provides the fastest response without overshoot. This result is expected since the LMI only considers nominal conditions, making the solution the least conservative among all controllers. In comparison with the LQR, the classic matching controller does not consider optimality; therefore, its solution resembles that of the LQR when $R = 0$. This explains why the control input amplitude at time $t = 0$ significantly violates the constraint, with amplitude $u = [5.2043 \quad -47.4587]^\top$.

- The robust matching controller, i.e., `method = 'robust'`, provides a response similar to the classic matching controller (the second-fastest response). This transient behavior is directly related to the LMI solved to compute the robust controller. Note that if the uncertainty set is larger, the resulting controller becomes more conservative, potentially leading to a slower transient response. The control input at time $t = 0$ also significantly violates the constraint, with amplitude $u = [4.4509 \quad -52.0066]^\top$.

- The constrained matching controller, i.e., `method = 'constrained'`, produces a less aggressive control input, faster than the LQR controller. This behavior results from the LMI solved to compute the controller, which imposes a constraint-admissible domain of attraction. Note, however, that constraints are still violated. It is important to emphasize that the constrained state-feedback gain can only guarantee constraint satisfaction within its domain of attraction. Since the initial conditions lie outside this domain, constraint violations occur. This limitation motivates the use of an MPC controller for systems that require hard constraint enforcement, as MPC can ensure constraint satisfaction even outside the nominal domain of attraction.

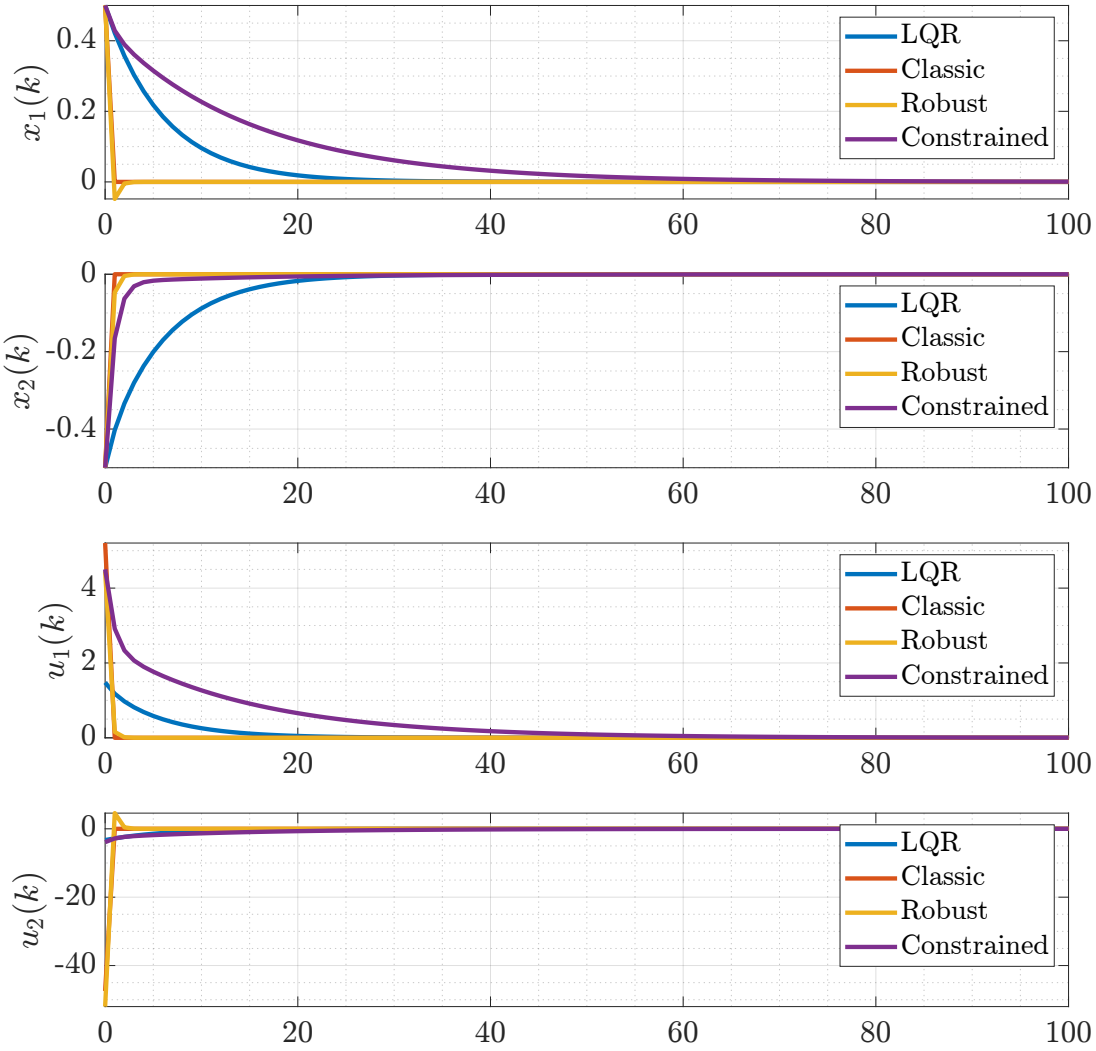


Figure 5: Simulations of the nominal (42), i.e., $\rho = 0_4$ and $\Delta A = 0_{2\times 2}$, in closed-loop with the different matching controllers (46).

8.2 Example: Robust MPC vs Classic MPC

This example shows a comparison of the different MPC performances classic MPC versus robust MPC. **In this case, we use the `MPCClass` class.**

Consider a generic discrete-time state-space realization, i.e., (42), where the uncertainty is represented by ΔA, i.e.,

$$\Delta A \in \mathbb{R}^{n\times n} : \Delta A := \begin{bmatrix} \rho_1 & \rho_2 \\ \rho_3 & \rho_4 \end{bmatrix} \quad \text{with} \quad (\rho_2, \rho_4) \in [-0.4, 0.4], \quad \text{and} \quad (\rho_1, \rho_3) = 0. \tag{49}$$

Consider the following constraints: $\mathbb{X} := \{x \in \mathbb{R}^2 : \|x\|_\infty \leq 1\}$ and $\mathbb{U} := \{u \in \mathbb{R}^2 : \|u\|_\infty \leq 10\}$.

The next excerpt of code shows how to solve a QP problem of MPC without and with robust tuning with various matching controllers for model (42).

```
% Example 2: Robust MPC vs Classic MPC with class
% Author: Victor Daniel Reyes Dreke

% CT-SS Realization. Remark: Non-minimum phase system
Ac =  [0, 1; -6, -5];
Bc =  [0, 0.1; 1, 0];
Cc =  [-1, 1];

% DT-SS Realization
Ts =  0.1;                          % [s] Sampling time
sysd = c2d(ss(Ac,Bc,Cc,[]),Ts);     % ZOH Discretization

% SS DT Matrices
A = sysd.A; B = sysd.B; C = sysd.C;

% Polytopic state and input constraints
Xbound = [[1; 1], [-1; -1]];            % [Xmax, Xmin]
Ubound = [[1; 1], [-1; -1]]*10;         % [Umax, Umin]
Pvec = [0, 0.4, 0, 0.4, zeros(1,4)];    % Pvec = [1 x nxm]

% MPC Hyper-parameters
N =  5;                                 % [N] prediction horizon
Q = eye(2); R = 1e-2*eye(2);            % Weigthing matrices

% Build object MPC
mpc_constrained = MPCClass(A,B,C,Pvec,Xbound,Ubound,N,Q,R); % Robust MPC
mpc_classic = MPCClass(A,B,C,Pvec,Xbound,Ubound,N,Q,R);     % Classic MPC

% Create OCP formulation
sigma = 10^-2;              % Hyper-parameters

% Tuning Methods => "robust", "constrained", "classic" (dlqr)
mpc_constrained = mpc_constrained.createOCP(sigma,"constrained",'sti','Hildreth');
mpc_classic = mpc_classic.createOCP(sigma,'dlqr','sti','Hildreth');

% Initial conditions
x0 = [0.5; -0.5];
u0 = zeros(2,1);

% Solve QP problem
u_qp_const = mpc_constrained.solveOneQP(x0,u0);
u_qp_clas = mpc_classic.solveOneQP(x0,u0);
```

To run the previous code, use:

Code

`RobCMMC_Exmp_MPCClass`

Figures 6-9 show the closed-loop response of system (42) controlled by two different MPC implementations. In this case, the simulations considers different values of $\rho \in \mathbb{P}$, instead that only considering nominal conditions like in the previous example. Note that the nominal dynamics with $\rho = 0_4$ are plotted in green, the uncertain dynamics are plotted using a palette color ranging from violet, i.e., $\rho = \rho_{max}$ to cyan, i.e., $\rho = \rho_{min}$.

Remark 8.2. *To have simulations results even when infeasibility issues are constrained violations are presented, we simulates the closed-loop behavior of the system allowing in case of infeasibility to use as control input the unconstrained MPC solution. This why, Figure 6-7 show state and input trajectories that violates constraints.*

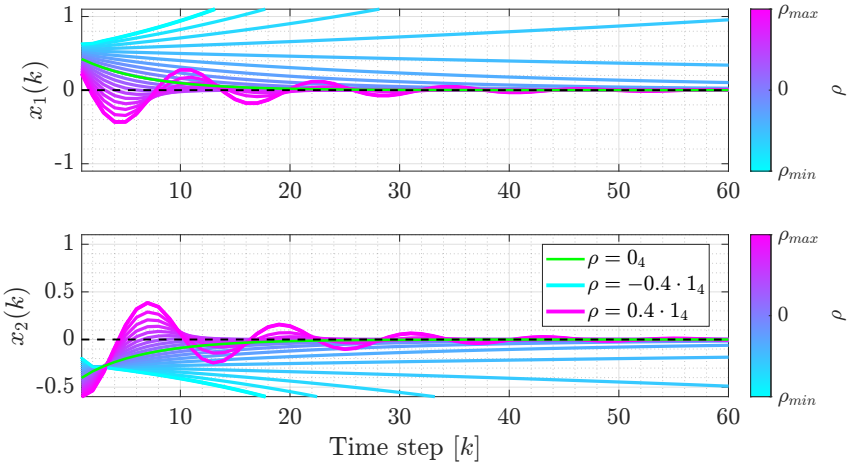


Figure 6: Closed-loop state trajectories of system (42) controlled by classic **MPC without automatic tuning** and varying ρ. The green trajectory represents the nominal response.

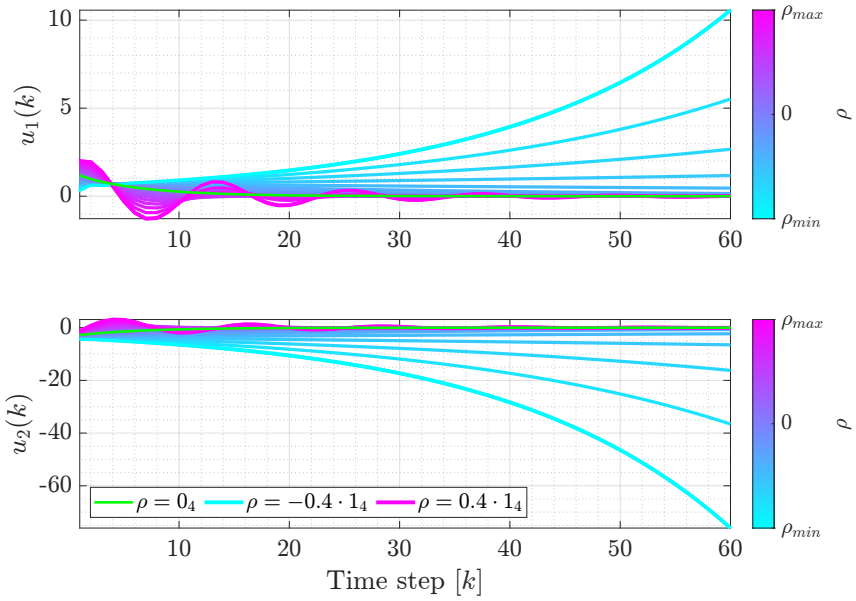


Figure 7: Closed-loop input trajectories of system (42) controlled by classic MPC without automatic tuning and varying ρ. The green trajectory represents the nominal response.

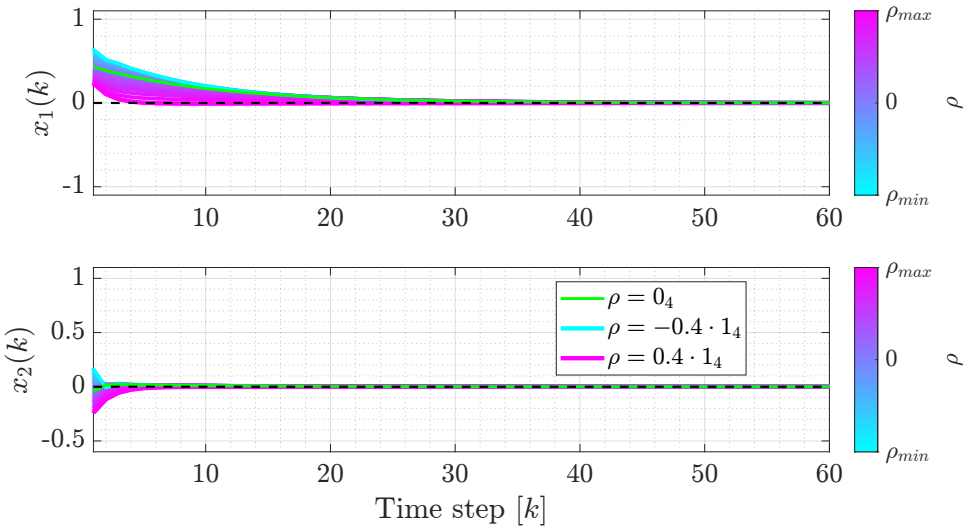


Figure 8: Closed-loop state trajectories of system (42) controlled by classic MPC with automatic tuning and varying ρ. The green trajectory represents the nominal response.

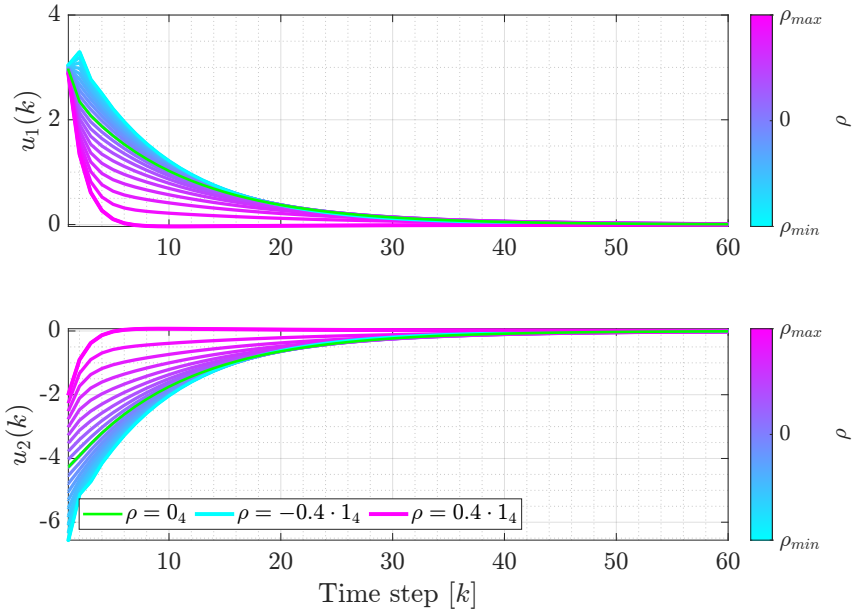


Figure 9: Closed-loop input trajectories of system (42) controlled by classic MPC with automatic tuning and varying ρ. The green trajectory represents the nominal response.

Observing Figure 6-7 that depict the closed-loop trajectory of the states and control inputs, respectively, when automatic tuning is not considered. We can confirm the relevance of having a robust tuned MPC. In this case, the Q, R assign empirically and the corresponding P computed the LQR methods cannot guarantee stability and recursive feasibility for all the possible dynamics determined by the uncertain matrices ρ. The worst results are observed when $\rho \to \rho_{min}$ because, the the zero in the right-half plane (RHP) vanished and the nature of the systems changes: it is no longer a non-minimum phase system as it is the nominal case. It is important to highlight that these are very critical changes in the systems dynamics that are not expected in the normal operation of MMCs.

Conversely, using the robust tuning method proposed in this toolbox we can obtain a desired closed-loop performance for all $\rho \in \mathbb{P}$. In this case, the the computed Q, R and P provides all the intended stability and feasibility guarantees.

8.3 Example: Modeling 2GW HVDC station from Tennet

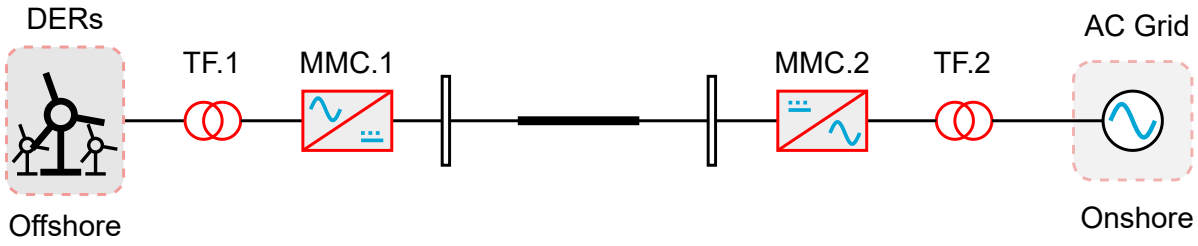


Figure 10: Simplified schematic of the one-terminal 2Gw Tennet station.

Rated parameters	MMC (MMC.2) Onshore	MMC (MMC.1) Offshore
L_r	0.2077 [p.u.]	0.4154 [p.u.]
R_r	0.0025 [p.u.]	0.0025 [p.u.]
L_{ac}	0.18 [p.u.]	0.15 [p.u.]
R_{ac}	0.006 [p.u.]	0.006 [p.u.]
f_o	50 Hz	50 Hz

Table 1: MMC's rated parameters.

The following excerpt of code shows how to derive the discrete-time state space representation given the per unit values of the MMC parameters.

```
% Example Tennet Station Modeling
% Author: Victor Daniel Reyes
% ------------------------------------------------------------------------ %

% Time parameters
Ts = 50e-6;

% Load MMC Parameters in struct variable. Note: This function is within the
% examples folders
MMC_ONSHORE  = getMMCAvgParameterPU('Tennet_2GW_Onshore', Ts);
MMC_OFFSHORE = getMMCAvgParameterPU('Tennet_2GW_Offshore',Ts);

% Building CT SS Representation
% Onshore MMC
[Ac_on, Bc_on, Cc_on] = getPlantModel(MMC_ONSHORE.Rr, MMC_ONSHORE.Rac, ...
                            MMC_ONSHORE.Lr, MMC_ONSHORE.Lac, ...
                            MMC_ONSHORE.w, 'OCC');
% Offshore MMC
[Ac_of, Bc_of, Cc_of] = getPlantModel(MMC_OFFSHORE.Rr, MMC_OFFSHORE.Rac, ...
                            MMC_OFFSHORE.Lr, MMC_OFFSHORE.Lac, ...
                            MMC_OFFSHORE.w, 'OCC');
% DT SS Representation
SYS_On_DISC = c2d(ss(Ac_on, Bc_on, Cc_on,[]),MMC_ONSHORE.Ts_pu,'zoh');
SYS_Of_DISC = c2d(ss(Ac_of, Bc_of, Cc_of,[]),MMC_OFFSHORE.Ts_pu,'zoh');

A_on = SYS_On_DISC.A; B_on = SYS_On_DISC.B;   C_on = SYS_On_DISC.C;
A_of = SYS_Of_DISC.A; B_of = SYS_Of_DISC.B;   C_of = SYS_Of_DISC.C;

% Simulation DT SS Representation using for loop
...
```

To run the previous code, use:

Code

```
RobCMMC_Exmp_TennetStat
```

Figure 11 depicts the open-loop responses of both MMCs when only nominal dynamics are considered. From Table 1, it can be observed that the two MMCs do not share the same rated parameters. These small differences in the rated parameters result in slight variations in the open-loop responses of the MMCs. In both cases, the MMCs exhibit stable dynamics. From a dynamic perspective, the main difference lies in the natural frequencies and gains of the systems.

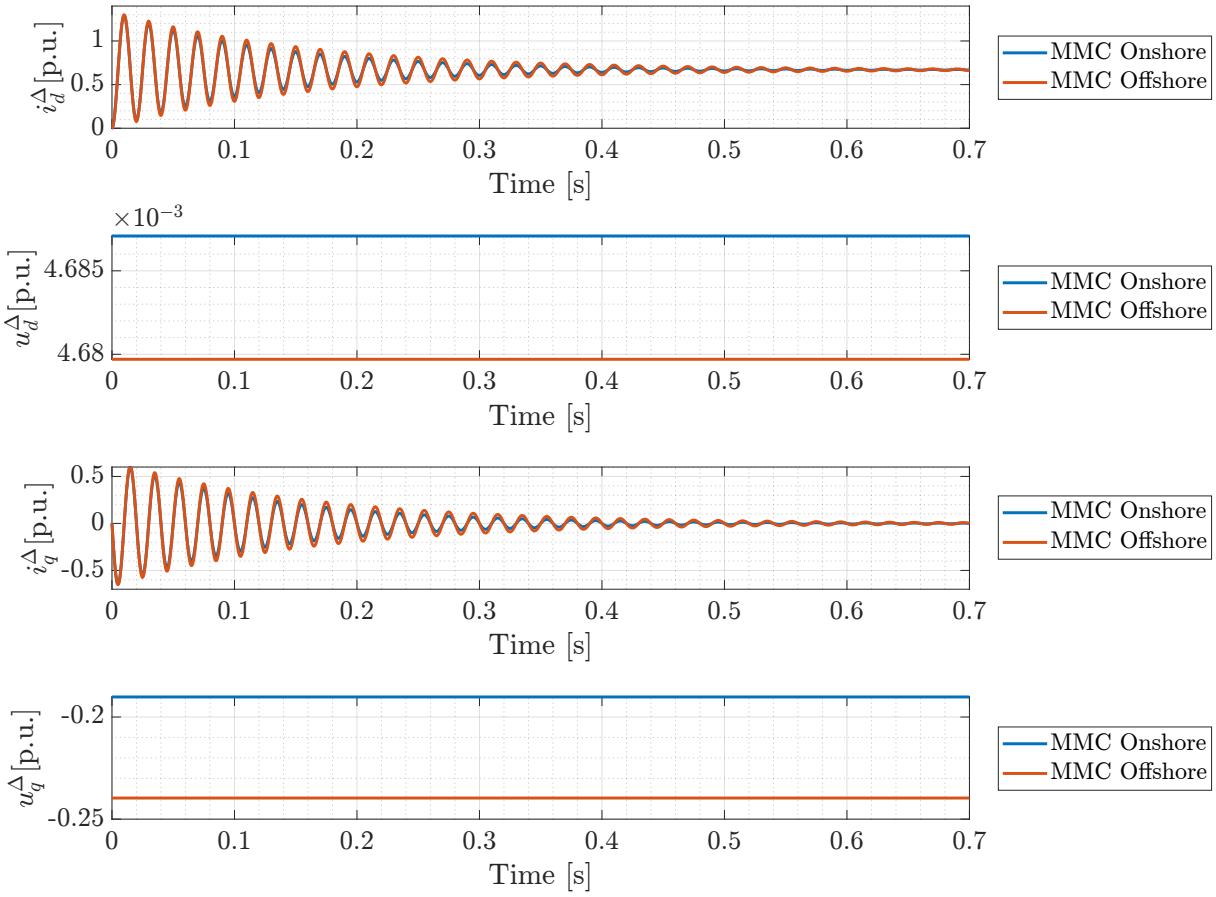


Figure 11: Step response of the MMC linear model of the output current.

8.3.1 Uncertain output current model

For the rated parameters in Table 1, we can derive a state-space representation of the MMC dynamics. In this example, we will consider structured and unstructured uncertainties. The structured uncertainties[2] are the following:

- Tolerance in the resistance values, i.e.,

$$R_r = R_r \pm \Delta R, \quad \text{and} \quad R_{ac} = R_{ac} \pm \Delta R, \quad \text{with} \quad \Delta R \in 80\% R_{ac}$$

- Variation of the nominal frequency, due to faults or grid-forming operation such that

$$f = f_0 \pm \Delta f, \quad \text{with} \quad \Delta f \in 30\% f_0.$$

With that, we can construct an uncertain state-space realization as in the mathematical background section, such that

$$\begin{aligned} \frac{d}{dt}x(t) &= (A_0 + \Delta A)x(t) + (B_0 + \Delta B)u(t) \\ y(t) &= Cx(t) \end{aligned} \tag{50}$$

where the values of nominal matrices can be obtain using the function `getPlantModel`$(\cdot)$. The uncertainty matrices are defined by

$$\Delta A \in \mathbb{R}^{n\times n} : \Delta A := \begin{bmatrix} \rho_1 & \rho_2 \\ \rho_3 & \rho_4 \end{bmatrix} \quad \text{and} \quad \Delta B \in \mathbb{R}^{n\times n} : \Delta B := \begin{bmatrix} \rho_5 & \rho_6 \\ \rho_7 & \rho_8 \end{bmatrix} \tag{51}$$

with

$$\begin{aligned} &\rho_1 \in [-3e^{-2}, 3e^{-2}], \quad \rho_2 \in [-5e^{-1}, 5e^{-1}], \quad \rho_3 \in [-5e^{-1}, 5e^{-1}], \quad \rho_4 \in [-3e^{-2}, 3e^{-2}] \\ &\rho_5 \in [-1e^{-4}, 1e^{-4}], \quad \rho_6 \in [-1e^{-4}, 1e^{-4}], \quad \rho_7 \in [-1e^{-4}, 1e^{-4}], \quad \rho_8 \in [-1e^{-4}, 1e^{-4}]. \end{aligned} \tag{52}$$

Note that range of the uncertainty coefficients consider the structured and unstructured uncertainties.

Figure 12 illustrates the open-loop response of the onshore MMC under different uncertainty parameter values. The parameters, properties, and trajectories corresponding to each element of the uncertainty-parameter set (σ) are represented using different colors: **System** 1 (red), **System** 2 (green), **System** 3 (cyan), and **System** 4 (violet).

In Figure 12, we observe the following:

- **System 1** $(\sigma = 1)$: The two poles are located at $p_{1,2} = 1.002 \pm 0.015j$. The output current trajectories under a constant voltage input exhibit unstable behavior. The primary cause of this instability is the presence of unstructured uncertainties.
- **System 2** $(\sigma = 2)$: The two poles are located at $p_{1,2} = 1.036 \pm 0.011j$. The output current trajectories under a constant voltage input exhibit unstable behavior. The primary cause of this instability is the presence of unstructured uncertainties.
- **System 3** $(\sigma = 3)$: The two poles are located at $p_{1,2} = 0.9914 \pm 0.013j$. The output current trajectories under a constant voltage input exhibit stable behavior. In this case, the system shows an overshoot of 48.09% and a settling time of 0.06875 s.
- **System 4** $(\sigma = 4)$: The two poles are located at $p_{1,2} = 0.9965 \pm 0.011j$. The output current trajectories under a constant voltage input exhibit stable behavior. In this case, the system shows an overshoot of 15.19% and a settling time of 0.06599 s.

Remark 8.3. *Structured uncertainties alone cannot render the system unstable; no feasible tolerance of the passive components can cause instability.*

[2]The tolerance values are not extracted from real case study. These are set for academic purpose and not to recreate real applications. I am still working on finding some real-time situations that can make the MMC system unstable :).

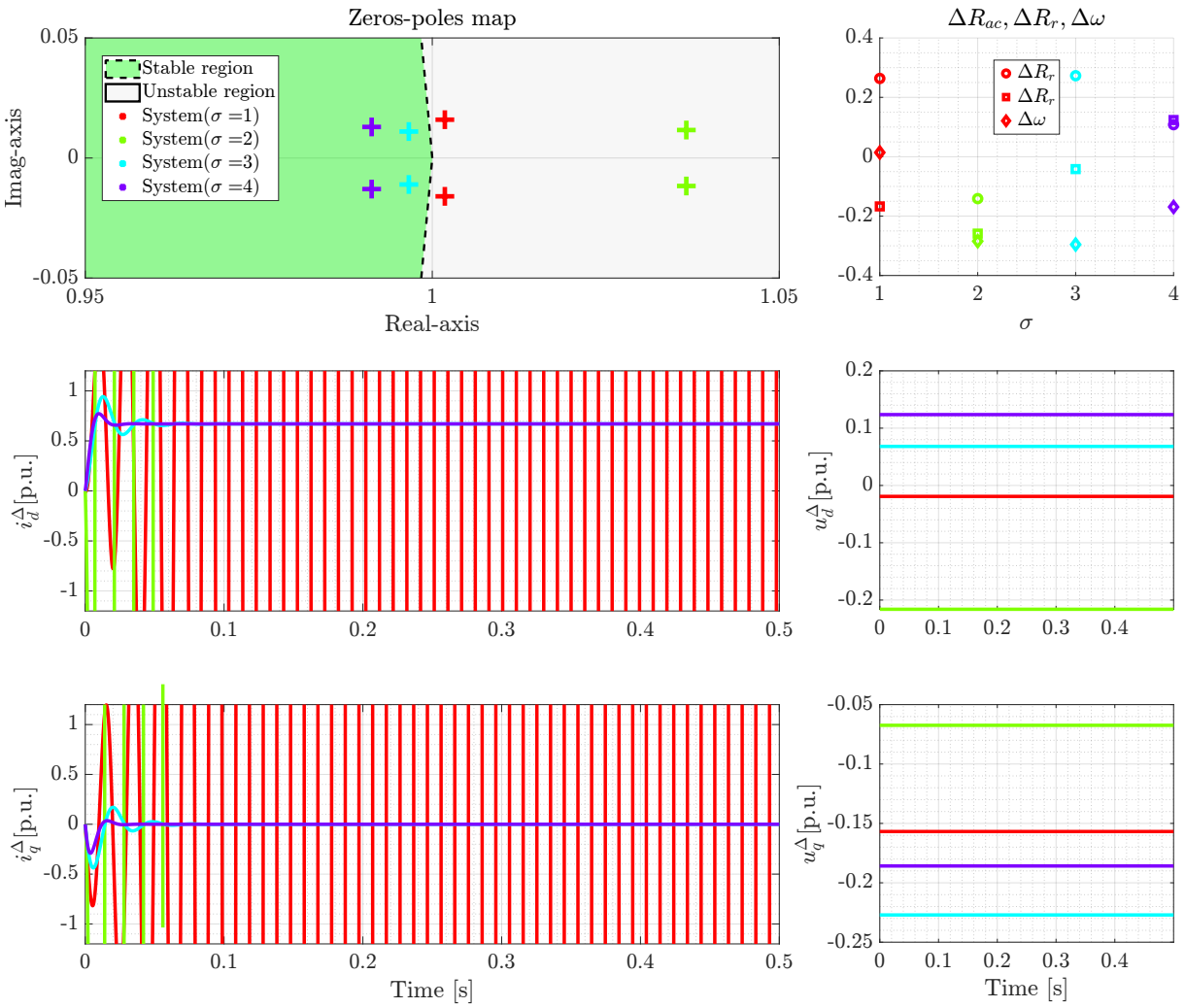


Figure 12: (Top left) Location of the open-loop poles of the onshore MMC dynamics, where the green region denotes the stability area in the z-plane. (Middle-bottom left) Open-loop trajectories of the output current in the dq reference frame under a constant step input.(Middle-bottom right) Constant trajectories of the control input. (Top right) Evaluated tolerance values and modeling uncertainties.

8.3.2 Output current controller design

Considering model (50), we design a output current controller based on **LQR** method. Furthermore, we design a **robust constrained** output current controller, i.e., (34), using model (50), scheduling vector (52), and design constraint sets $\mathbb{X} \subseteq \mathbb{R}^2_{[-5,5]}$ and $\mathbb{U} \subseteq \mathbb{R}^2_{[-0.2,0.2]}$. The following code is an example of how to implemented this solution.

```
% example_tennet_closed_loop_dynamics
% Author: Victor Daniel Reyes
% ------------------------------------------------------------------------ %
close all, clc,
addpath lib\
% Time parameters
Ts = 50e-6;     % [s] sampling time

% Load per-unit [p.u] MMC Parameters in a struct variable.
MMC_ONSHORE  = getMMCAvgParameterPU('Tennet_2GW_Onshore', Ts);  % [p.u]
% Building CT SS Representation: Onshore MMC
[Ac_on, Bc_on, Cc_on] = getPlantModel(MMC_ONSHORE.Rr, MMC_ONSHORE.Rac, ...
                            MMC_ONSHORE.Lr, MMC_ONSHORE.Lac, ...
                            MMC_ONSHORE.w, 'occ');
% DT SS Representation
SYS_On_DISC = c2d(ss(Ac_on, Bc_on, Cc_on,[]),MMC_ONSHORE.Ts_pu,'zoh');

% Rearrangement
A_on = SYS_On_DISC.A;  B_on = SYS_On_DISC.B; C_on = SYS_On_DISC.C;

% Model uncertaint coefficient
NModel =  4;                      % Number of cases
% Edit this line to modify tolerance values ranges. Default: -0.5 to 0.5
prv_max =  [6e-1,...       % \Delta R_r,
            6e-1,...       % \Delta R_ac,
            4e-1,...       % \Delta L_r
            4e-1,...       % \Delta L_ac
            6e-1];         % \Delta w
prv = zeros(length(prv_max),NModel);  % Vector of uncertaint coefficient

for i=1:NModel
    for j=1:size(prv,1)
        prv(j,i)= (rand(1,1)-0.5)*prv_max(j);       %
    end
end

% Polytopic state and input constraints
Xbound = [[1; 1], [-1; -1]]*5;                  % [Xmax, Xmin]
Ubound = [[1; 1], [-1; -1]]*0.2;                % [Umax, Umin]

Pvec = [2e-2, 1.5e-1, 1.5e-1, 2e-2, zeros(1,4)];    % Pvec = [1 x nxm]

%
%  Controller Design Case 1: LQR
Q = 1e2*eye(2); R =1e-6*eye(2);
K_LQR = dlqr(A_on,B_on,Q,R);
%  Controller Design Case 2: Constrained
[K_Const,P_Const]=getRobustK(A_on,B_on,[],Pvec,Xbound,Ubound,"constrained");

% Simulation DT SS Representation using for loop
...
```

Running this code yields the following matching controllers:

$$\texttt{K_LQR} = \begin{bmatrix} -18.0668 & 0.1419 \\ -0.1419 & -18.0668 \end{bmatrix}, \quad \texttt{K_Const} = \begin{bmatrix} -3.1000 & 0.1855 \\ 0.1349 & -3.0918 \end{bmatrix}. \tag{53}$$

Figure 13 illustrates the closed-loop response of the onshore MMC under different uncertainty parameter values when $\texttt{K}_{\texttt{LQR}}$ is used. The parameters, properties, and trajectories corresponding to each element of the uncertainty-parameter set (σ) are represented using different colors: **System** 1 (red), **System** 2 (green), **System** 3 (cyan), and **System** 4 (violet).

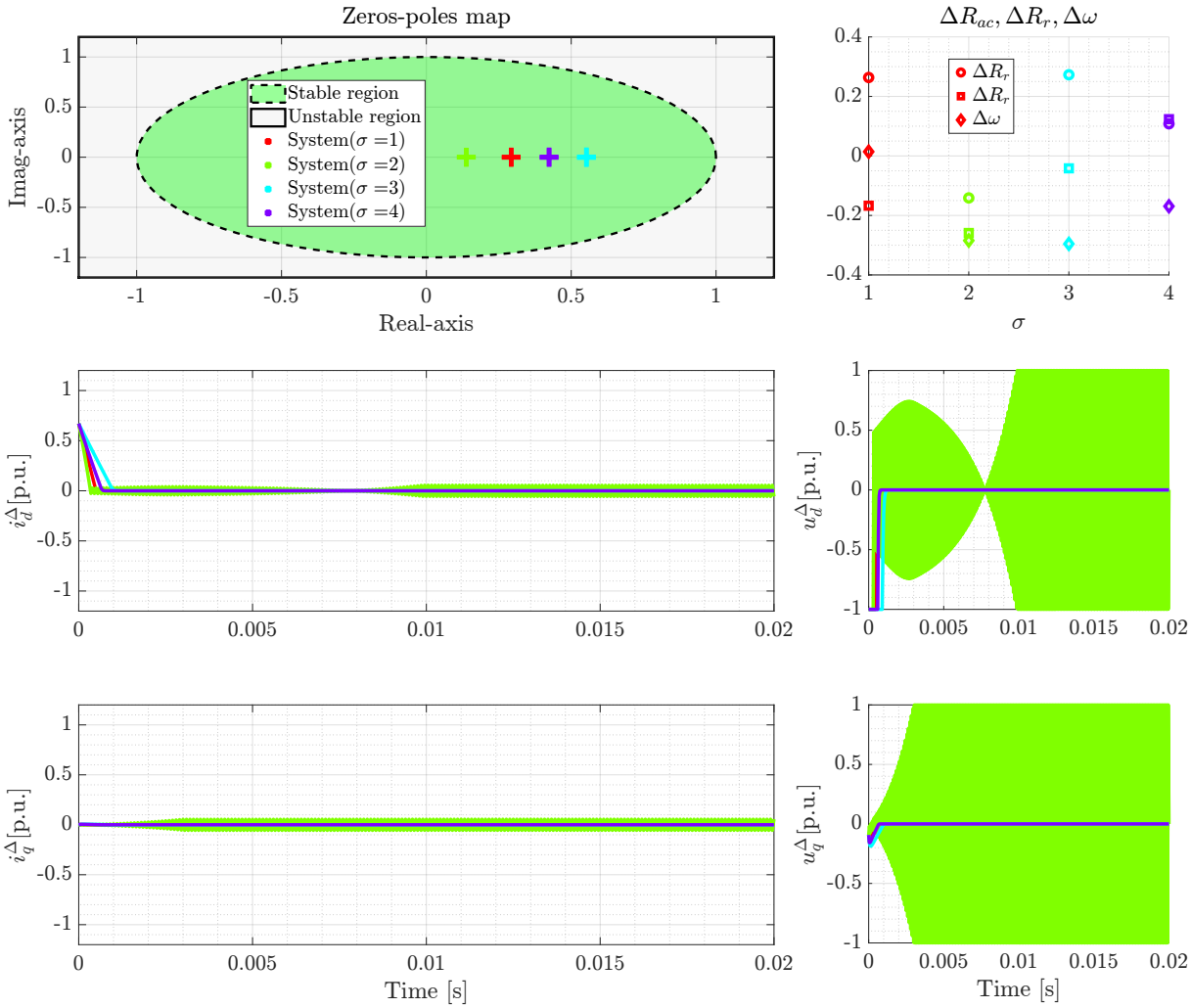


Figure 13: (Top left) Location of the closed-loop poles of the onshore MMC dynamics when classic LQR controller (K_LQR) is used, where the green region denotes the stability area in the z-plane. (Middle-bottom left) Closed-loop trajectories of the output current in the dq reference frame under a constant step input.(Middle-bottom right) Closed-loop trajectories of the control input. (Top right) Evaluated tolerance values and modeling uncertainties.

In Figure 13, we observe the following:

- **System 1** ($\sigma = 1$): The two poles are located at $p_{1,2} = 0.293 \pm 0.002j$. The output current trajectories under a constant voltage input exhibit stable behavior. The system shows no overshoot and achieves a settling time of 0.5 ms (i.e., $10T_s$). The control inputs are saturated during the first 0.1 ms of the simulation.
- **System 2** ($\sigma = 2$): The two poles are located at $p_{1,2} = 0.1378 \pm 5 \times 10^{-4}j$. The output current trajectories under a constant voltage input exhibit stable behavior. The system

shows no overshoot; however, due to input saturation, it reaches a limit cycle with a current ripple of 0.2687 [p.u.]. The corresponding control input also reaches a limit cycle with a voltage ripple of 1 [p.u.].

- **System 3** ($\sigma = 3$): The two poles are located at $p_{1,2} = 0.5526 \pm 0.004j$. The output current trajectories under a constant voltage input exhibit stable behavior. The system shows no overshoot and achieves a settling time of 1.2 ms (i.e., $24T_s$). The control inputs are saturated during the first 0.1 ms of the simulation.

- **System 4** ($\sigma = 4$): The two poles are located at $p_{1,2} = 0.4235 \pm 0.003j$. The output current trajectories under a constant voltage input exhibit stable behavior. The system shows no overshoot and achieves a settling time of 0.75 ms (i.e., $15T_s$). The control inputs are saturated during the first 0.1 ms of the simulation.

Figure 14 illustrates the closed-loop response of the onshore MMC under different uncertainty parameter values, when `K_Const` is used. The parameters, properties and trajectories corresponding to each element of the set of uncertainty parameters (σ) are represented colors, such as **System** 1 (red), **System** 2 (green), **System** 3(cyan) and **System** 4 (violet).

In Figure 14, we observe the following:

- **System 1** ($\sigma = 1$): The two poles are located at $p_{1,2} = 0.812 \pm 0.0074j$. The output current trajectories under a constant voltage input exhibit stable behavior. The system shows no overshoot and achieves a settling time of 1.05 ms (i.e., $22T_s$).

- **System 2** ($\sigma = 2$): The two poles are located at $p_1 = 0.7221$ and $p_2 = 0.7453$. The output current trajectories under a constant voltage input exhibit stable behavior. The system shows no overshoot and achieves a settling time of 0.7 ms (i.e., $12T_s$).

- **System 3** ($\sigma = 3$): The two poles are located at $p_{1,2} = 0.901 \pm 0.008j$. The output current trajectories under a constant voltage input exhibit stable behavior. The system shows no overshoot and achieves a settling time of 2.2 ms (i.e., $42T_s$).

- **System 4** ($\sigma = 4$): The two poles are located at $p_{1,2} = 0.857 \pm 0.008j$. The output current trajectories under a constant voltage input exhibit stable behavior. The system shows no overshoot and achieves a settling time of 1.45 ms (i.e., $26T_s$).

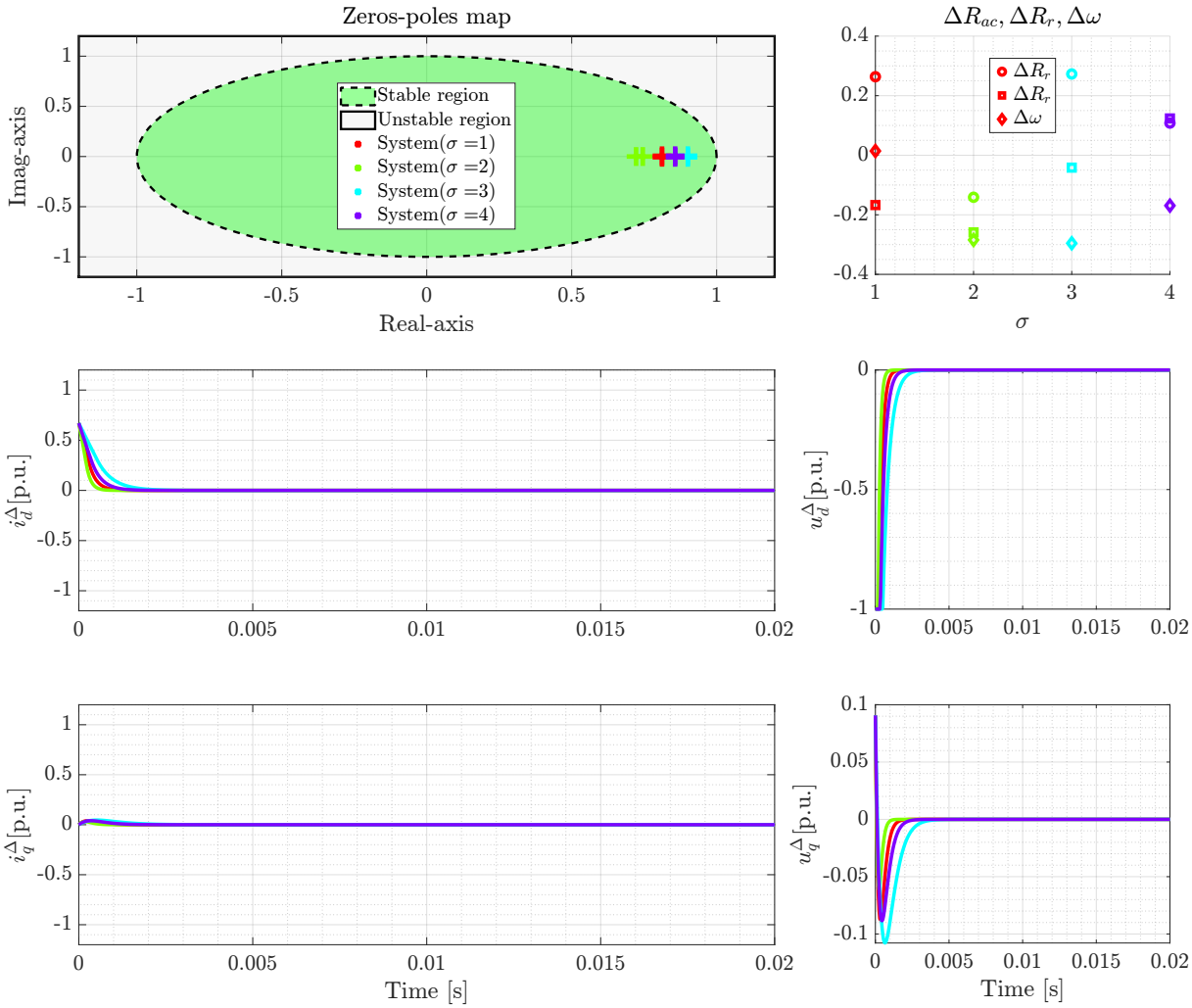


Figure 14: (Top left) Location of the closed-loop poles of the onshore MMC dynamics when proposed robust constrained controller (`K_Const`) is used, where the green region denotes the stability area in the z-plane. (Middle-bottom left) Closed-loop trajectories of the output current in the dq reference frame under a constant step input.(Middle-bottom right) Closed-loop trajectories of the control input. (Top right) Evaluated tolerance values and modeling uncertainties.

8.4 Example: Unconstrained control of 2GW HVDC station from Tennet

```
% Example Tennet Station Control
% Author: Victor Daniel Reyes
% ------------------------------------------------------------------------ %

... Time parameters

nx = size(A_on,1);
nu = size(B_on,2);
np = size(C_on,1);

% Current and Voltage limits
IBound = [-1.05, 1.05];
VBound = [-1.05, 1.05]/2;

% Polytopic constraints the system [(), 1]
XBOUNDS = kron(ones(nx,1),IBound);
UBOUNDS = kron(ones(nu,1),VBound);

% rho vector (scheduling variables) defintion [(), 1]
pvec = 2e-4*ones(1,8);

% LQR Hyper-parameters
Q = eye(2); R = 1e-2*eye(2);            % Weigthing matrices

% LQR Controller synthesis
[K,P_LQR] = dlqr(A_on,B_on,Q,R);  K_LQR = -K;


% Controller synthesis function. Choose "classic","robust" or "constrained".
[K,P]=getRobustK(A_on,B_on,[],pvec,XBOUNDS,UBOUNDS,"constrained");

% Simulation DT SS Representation using for loop
...
```

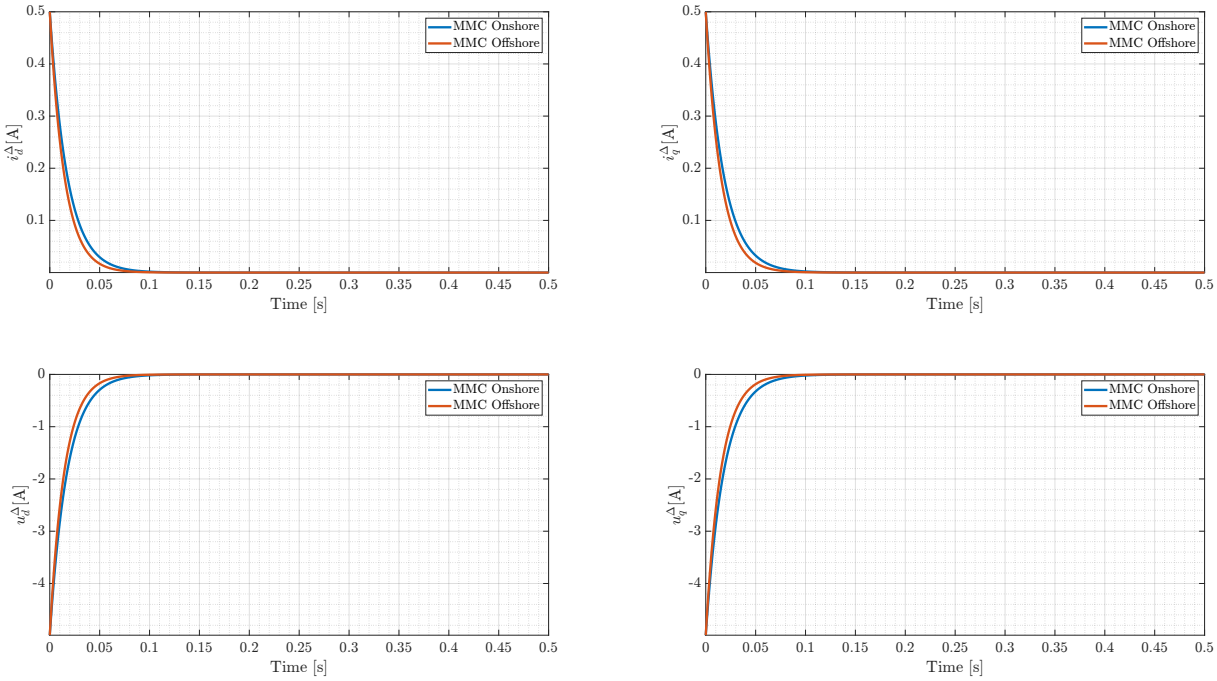


Figure 15: Comparison OCC closed-loop response both MMCs controlled by lqr state feedback gain.

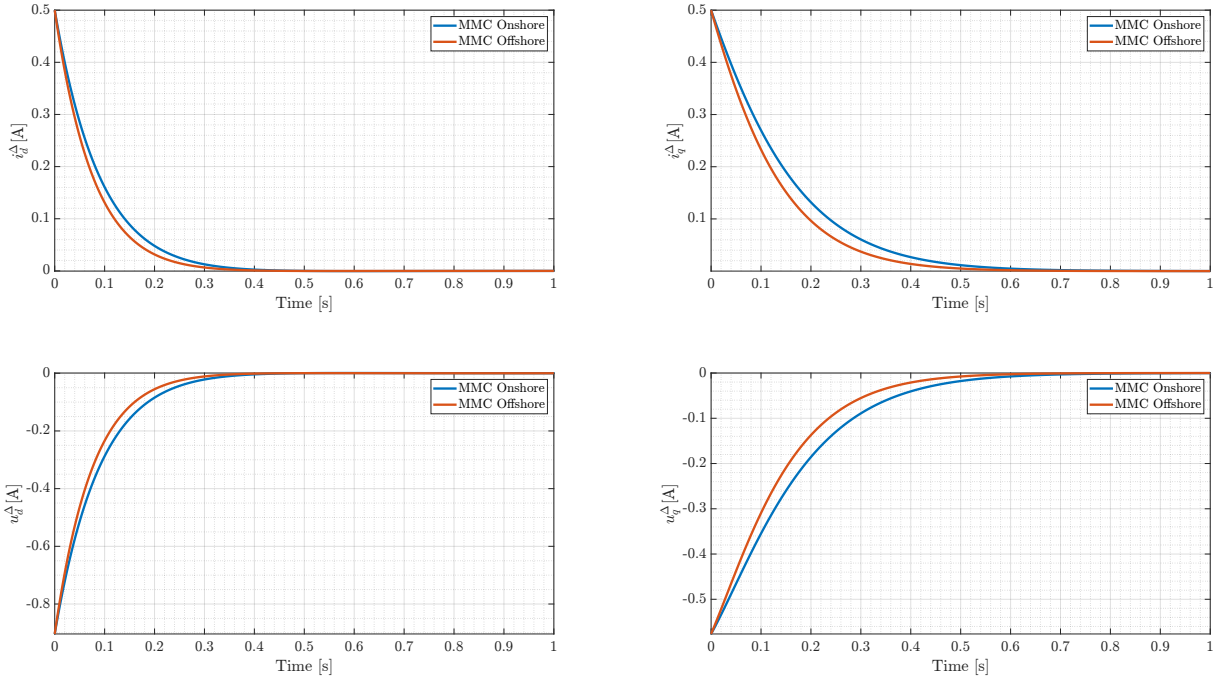


Figure 16: Comparison OCC closed-loop response both MMCs controlled by robust state feedback gain.

Figure 17 shows the trajectory of the output current controlled with the rMPC.

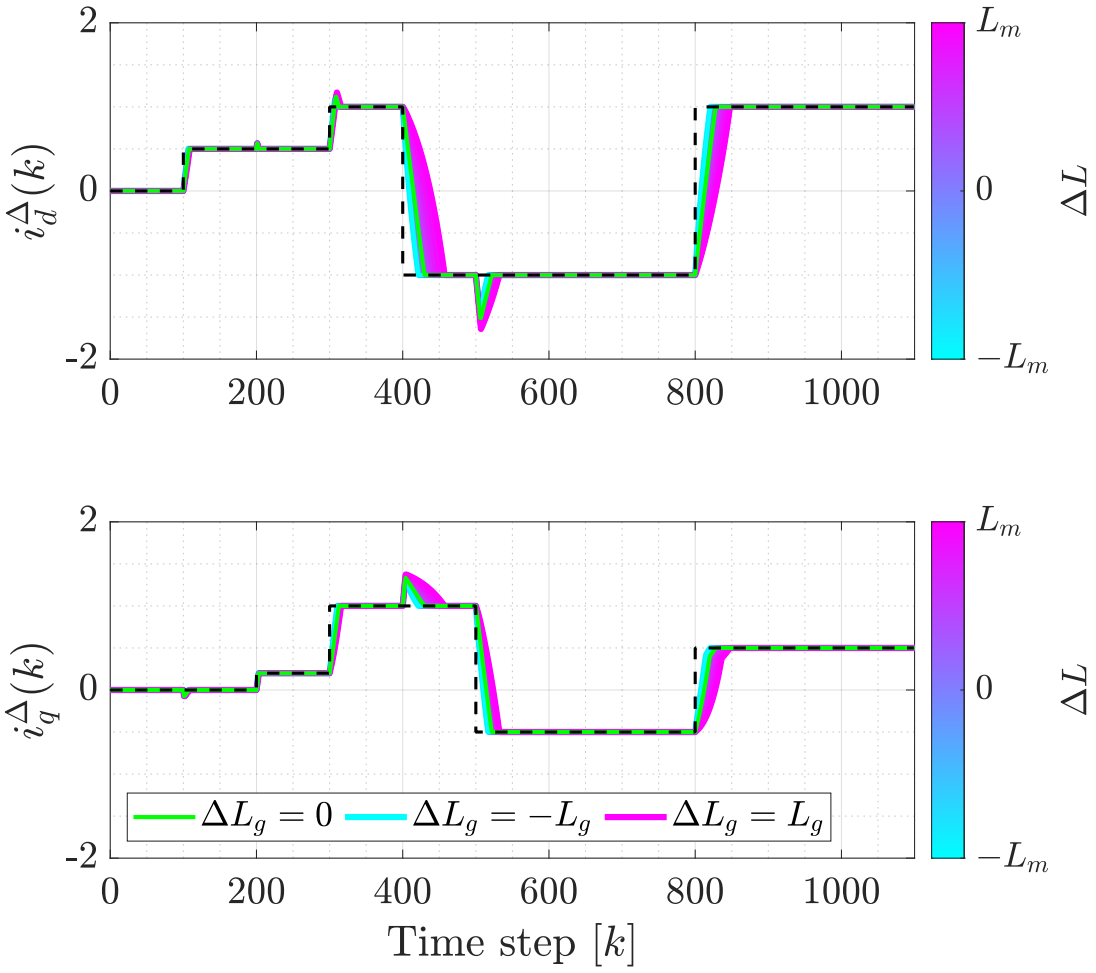


Figure 17: Trajectory of the MMC output current controlled with the rMPC.

Figure 18 shows the trajectory of the output voltage generated by the rMPC.

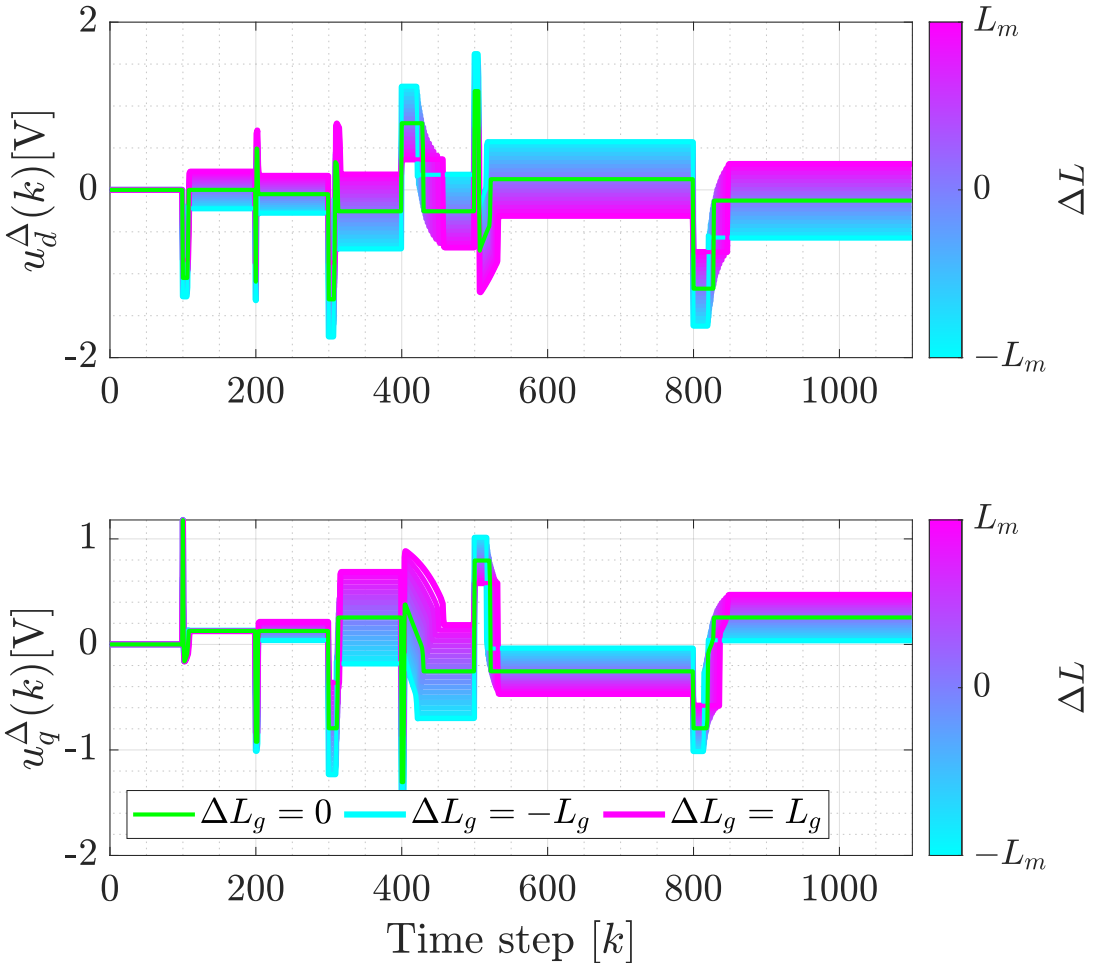


Figure 18: Trajectory of the MMC output voltage generated with the rMPC.

9 Appendix A: Basic flowchart of the workflow of the toolbox

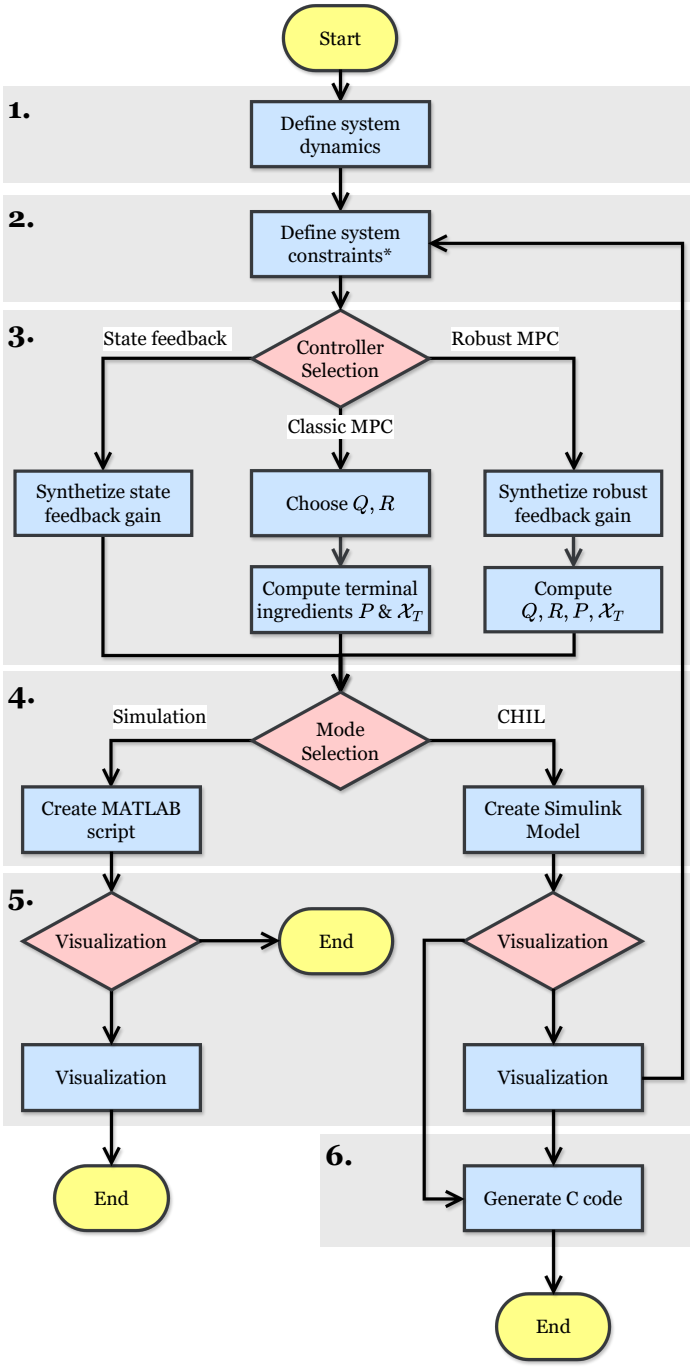


Figure 19: Flow chart of the basic workflow of the toolbox

10 References

References

[1] M. Ayari, M. M. Belhaouane, X. Guillaud, and N. B. Braiek, "Multivariable optimal robust control strategy for mmc converter," in *2017 International Conference on Advanced Systems and Electric Technologies*. IEEE, 2017, pp. 210–215.

[2] M. M. Belhaouane, M. Ayari, X. Guillaud, and N. B. Braiek, "Robust control design of mmc-hvdc systems using multivariable optimal guaranteed cost approach," *IEEE Transactions on Industry Applications*, vol. 55, no. 3, pp. 2952–2963, 2019.

[3] J. Löfberg, "Yalmip : A toolbox for modeling and optimization in matlab," in *In Proceedings of the CACSD Conference*, Taipei, Taiwan, 2004.

[4] M. Herceg, M. Kvasnica, C. Jones, and M. Morari, "Multi-Parametric Toolbox 3.0," in *Proc. of the European Control Conference*, Zürich, Switzerland, July 17–19 2013, pp. 502–510, http://control.ee.ethz.ch/~mpt.

[5] A. Shetgaonkar, L. Liu, A. Lekić, M. Popov, and P. Palensky, "Model predictive control and protection of mmc-based mtdc power systems," *International Journal of Electrical Power & Energy Systems*, vol. 146, p. 108710, 2023.

[6] V. {Reyes Dreke} and M. Lazar, "Practical deadbeat mpc design via controller matching with applications in power electronics," *European Journal of Control*, vol. 74, Nov. 2023.

[7] M. Lazar, "Lecture notes in model predictive control course," February 2021.

[8] L. Wang, *Model predictive control system design and implementation using MATLAB®*. Springer, 2009.